Breaking the Memory Wall: A Survey of Key-Value (KV) Cache Compression for Efficient Large Language Model (LLM) Inference

Rohith Reddy B. C.

[Department], [Institution], [City], [Country]

Corresponding author: rohithreddybc98@gmail.com · ORCID: 0000-0000-0000-0000

Abstract

The deployment of large language models (LLMs) at long context lengths is increasingly limited by memory rather than compute. During autoregressive decoding, the key-value (KV) cache, which stores the attention states of every processed token, grows linearly with sequence length and batch size and must be streamed in full for each generated token, making decoding memory-bandwidth-bound and constituting the dominant bottleneck of modern inference, a manifestation of the long-observed memory wall. This survey gives a unified, hardware-aware treatment of techniques that mitigate this bottleneck. We formalise it through the roofline model and a VRAM-footprint analysis separating compute-bound prefill from memory-bound decoding, then organise the literature into four complementary layers: algorithmic compression (quantization, eviction and sparsification, and token, low-rank, or learned merging); architectural redesign (multi-query and grouped-query attention, low-rank latent attention, and recurrent or hybrid state-space models); system-level management (paged memory, prefix sharing, cross-request transport, and tiered offloading); and hardware acceleration (decoding kernels, fusion, and processing-in-memory). For each we give the governing mechanism, achievable memory reduction, and accuracy-latency trade-offs. Distinct from prior surveys, we (a) unify the four layers under a co-design and Pareto-frontier framework, (b) consolidate the evidence on multi-tenant KV cache security and side-channel leakage, and (c) analyse cache degradation in long-horizon agentic loops. To address the benchmarking deficit that obscures progress, we propose Matched-Budget Evaluation (MBE), a standardised reporting protocol, released with an open harness, under which methods are compared at fixed memory budgets. It targets researchers and engineers combining KV cache optimisations under deployment constraints.

Keywords

Large language models · Key-value cache · Inference efficiency · Memory wall · Quantization · Attention mechanisms · Processing-in-memory

1. Introduction

1.1 The growth of context windows in large language models

Over the past decade, sequence modelling for natural language processing has undergone a sequence of architectural transitions, each of which has reshaped the dominant performance bottleneck. Early sequence-to-sequence models based on recurrent and long short-term memory networks compressed an input sequence into a single fixed-dimensional context vector. This recurrence imposed a strict sequential dependency and limited the ability of the network to represent long-range dependencies. The transformer architecture removed this dependency by replacing recurrence with global self-attention (Vaswani et al. 2017), allowing any two tokens to interact directly regardless of their distance.

Self-attention introduced a different constraint. The dense attention operation has quadratic time and memory complexity in the sequence length, and early models such as BERT (Devlin et al. 2019) and the first GPT models restricted the context window to a few hundred tokens. Subsequent algorithmic and systems advances, together with the demands of retrieval-augmented generation, document analysis, and agentic applications, together with positional-extrapolation techniques (Peng et al. 2024; Ding et al. 2024), have extended production context windows to hundreds of thousands of tokens. This expansion enables new applications but imposes severe constraints on inference hardware, because the attention states of all previously processed tokens must be retained for reuse during generation.

1.2 The memory wall in generative inference

Autoregressive transformer inference is best understood as two regimes with very different hardware behaviour: prefill and decoding. The prefill phase processes the input prompt in a single parallel forward pass dominated by dense matrix-matrix multiplications. These operations have high arithmetic intensity, map efficiently onto the tensor cores of modern accelerators, and approach the compute-bound roofline of the device.

Decoding behaves differently. Because each generated token is conditioned on all previous tokens, the dense matrix-matrix products of prefill become matrix-vector products with a query length of one. The arithmetic intensity collapses, and the bottleneck shifts from compute to off-chip memory bandwidth. This regime is the memory wall: the long-standing trend, first named by Wulf and McKee 1995 and revisited for AI workloads by Gholami et al. 2024, in which compute throughput scales far faster than memory bandwidth, leaving processing units stalled while data streams from memory (Williams et al. 2009). For decoding, generation latency is governed by physical memory bandwidth rather than peak floating-point throughput.

1.3 The key-value cache bottleneck

The key-value (KV) cache is the central object of this memory-bandwidth bottleneck. Rather than recomputing the key and value projections of all past tokens at every step, which would incur quadratic cost, inference engines cache these tensors and append the projections of each new token as it is generated, reducing per-step attention cost from quadratic to linear (Kwon et al. 2023). The trade-off is a cache whose size grows linearly with context length, batch size, layer count, and head dimension. For frontier models the dynamic cache can rival or exceed the static

weight footprint, and a single long-context request can consume tens of gigabytes of high-bandwidth memory (Hooper et al. 2024a).

The operational consequences are direct. Throughput-oriented serving batches many requests together, but the aggregate KV cache then forces small batch sizes; exceeding the available memory triggers out-of-memory failures or preemption, lowering throughput and raising cost per token. In multi-tenant deployments the cache becomes the primary constraint on cluster economics (Kwon et al. 2023; Li et al. 2025a).

1.4 Relationship to existing surveys

Several recent surveys address KV cache optimisation, and we state our distinct contribution relative to them. The most comprehensive is Li et al. 2025a, which organises methods into token-level, model-level, and system-level optimisations and catalogues datasets and benchmarks; it explicitly centres computational and memory efficiency and does not treat multi-tenant security or long-horizon agentic degradation. Shi et al. 2024 review methods across the pre-training, deployment, and inference phases and summarise long-text evaluation metrics. Liu et al. 2025c provide a shorter review focused on selective-token, quantization, and attention-compression strategies. Wolters et al. 2024 survey compute-in-memory hardware for transformers but do not cover algorithmic or system-level compression in depth, while Zhang et al. 2024e survey agent memory mechanisms without a hardware or KV-cache focus. The most recent entrants confirm rather than close these gaps: Zhen et al. 2025 survey efficient inference serving broadly; Xu et al. 2026 organise methods into eviction, compression, hybrid memory, novel attention, and combination strategies and map them to deployment scenarios; and Jiang et al. 2026 adopt a system-behaviour (temporal, spatial, structural) taxonomy of serving-time methods. None of these treats multi-tenant KV cache security or long-horizon agentic degradation, and, critically, none proposes a standardised evaluation protocol. Beyond KV-cache-specific reviews, broader surveys of efficient inference (Zhou et al. 2024b; Wan et al. 2024; Miao et al. 2023), long-context modelling (Liu et al. 2025a), efficient attention (Sun et al. 2025), and the hardware perspective (Li et al. 2024a) provide useful context; the roofline-based analysis of Yuan et al. 2024b is closest in spirit to our hardware framing, though it treats LLM inference broadly rather than centring the KV cache. Table 1 positions the present survey against the KV-cache-specific works.

This survey is differentiated along four axes. First, it adopts a hardware-first organisation in which every algorithmic, architectural, and system technique is tied back to the roofline and memory-traffic analysis of Section 3, and it treats the four optimisation layers under a single co-design and Pareto-frontier framework that makes the interference between stacked methods explicit (Section 8). Second, it consolidates the rapidly growing evidence on KV cache security in multi-tenant serving, including prefix-sharing side channels and prompt-leakage attacks, which prior KV-cache surveys omit (Sections 6.2 and 9.4). Third, it analyses degradation modes specific to long-horizon agentic loops and the benchmarking deficit that conceals them (Sections 9.2 and 9.3). Fourth, and uniquely among surveys in this area, it converts that benchmarking critique into a concrete remedy: the Matched-Budget Evaluation protocol and its open harness (Section 10), so that the survey contributes an adoptable artifact rather than only a synthesis.

Table 1. Positioning of this survey against existing reviews of KV cache and inference efficiency. No prior survey jointly covers multi-tenant security and agentic degradation, and none proposes a standardised evaluation protocol (the right-most column).

Survey (year)	Primary taxonomy / angle	HW / PIM	Co-design	Security	Agentic	Eval protocol
Li et al. 2025a	Token / model / system	Medium	Partial	No	No	No (datasets listed)
Shi et al. 2024	Train / deploy / infer phase	Low	No	No	No	No (metrics listed)
Liu et al. 2025c	Token / quant / attention	Low	No	No	No	No
Zhen et al. 2025	Inference-serving (broad)	Medium	No	No	No	No
Xu et al. 2026	Eviction / compress / hybrid / attention	No	Partial	No	No	No
Jiang et al. 2026	Temporal / spatial / structural (serving)	High	Partial	No	No	No
Wolters et al. 2024	Compute-in-memory hardware	High	No	No	No	No
Zhang et al. 2024e	Agent memory mechanisms	None	No	No	Yes	No (agent tasks)
This survey (2026)	Algorithmic / architectural / system / hardware (4-layer)	High	Yes	Yes	Yes	Yes (MBE)

1.5 Contributions and organisation

The contributions of this survey are: (1) a unified, hardware-grounded taxonomy of KV cache optimisation spanning algorithmic compression, architectural redesign, system-level memory management, and hardware acceleration, with a consistent set of evaluation axes (memory reduction, accuracy impact, deployment prerequisite, and bottleneck addressed); (2) a co-design and Pareto-frontier analysis that characterises how orthogonal techniques interact, including the algorithmic-mathematical interference that arises when eviction and quantization are naively stacked; (3) a consolidated treatment of two under-surveyed but deployment-critical concerns, multi-tenant KV cache security and long-horizon agentic degradation, together with the benchmarking gaps that hinder progress; and (4) Matched-Budget Evaluation (MBE), a concrete standardised protocol and open harness for reporting KV cache compression results at fixed memory budgets, intended to make the field's results comparable (Section 10).

The remainder of the survey is organised as follows. Section 2 describes the review methodology and scope. Section 3 establishes the theoretical foundations of the KV cache and the prefill/decoding divide. Section 4 surveys algorithmic compression: quantization, eviction and sparsification, and structural merging. Section 5 covers architectural paradigm shifts in attention and sequence modelling. Section 6 addresses system-level memory management, including paging, prefix sharing with its security implications, and tiered offloading. Section 7 treats hardware-level acceleration. Section 8 presents a comparative synthesis, co-design analysis, and

Pareto frontiers. Section 9 identifies open challenges. Section 10 specifies the Matched-Budget Evaluation (MBE) protocol, and Section 11 concludes.

2. Review Methodology and Scope

This survey follows a structured protocol so that its coverage is reproducible and its boundaries are explicit. We targeted peer-reviewed and archival literature on KV cache memory reduction and the architectures and systems that support it, published primarily between 2019 and 2026, while retaining seminal earlier work on attention and the roofline model for context.

Sources were identified through three complementary channels, with a search cut-off of June 2026: (i) keyword and semantic search of scholarly aggregators indexing Semantic Scholar, arXiv, and Scopus; (ii) a full-text peer-reviewed corpus; and (iii) forward and backward citation tracing from the most cited methods and from the surveys in Table 1. Search terms combined the concepts of KV cache, attention cache, quantization, eviction, sparsity, paged attention, latent attention, state-space models, processing-in-memory, and long-context inference. We prioritised publications at machine-learning, systems, and architecture venues (for example NeurIPS, ICML, ICLR, ACL, MLSys, OSDI, SOSP, ASPLOS, and ISCA) and peer-reviewed journals, supplemented by influential preprints that introduced widely adopted methods. Figure 1 summarises the selection flow: approximately 612 candidate records were identified, reduced to about 318 after de-duplication and title screening, about 206 assessed in full text for eligibility and venue quality, and 178 retained for detailed analysis and inclusion. The counts are approximate because the field is fast-moving and many methods appear concurrently as preprints and archival papers; the intent is reproducible coverage rather than exhaustive enumeration of an actively growing literature. Of the 198 references in this survey, 178 are KV-cache methods or systems analysed in depth; the remaining 20 are seminal or background works (for example, foundational attention, the memory wall, and the competing surveys) cited for context.

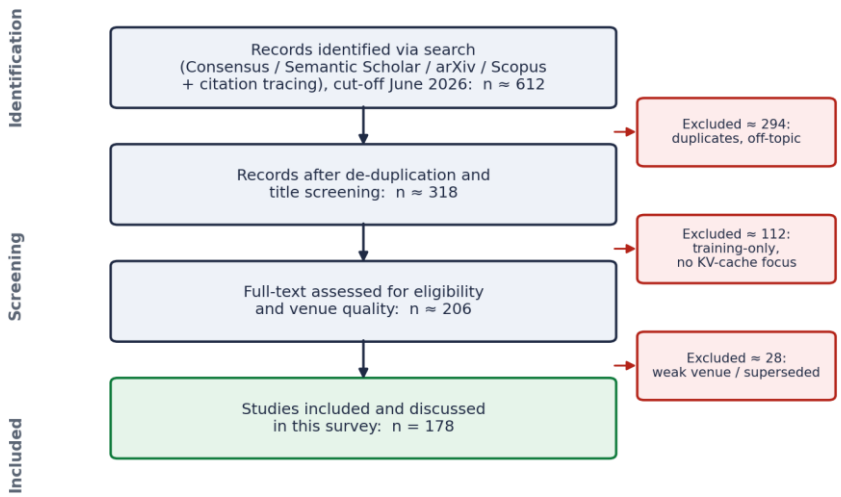

Fig. 1. PRISMA-style flow of identification, screening, and inclusion for the survey (counts approximate).

Inclusion required that a work either (a) reduces the memory or bandwidth cost of the KV cache, (b) redesigns the attention or sequence-modelling mechanism to bound that cost, (c) manages KV cache memory at the system or hardware level, or (d) evaluates the accuracy, latency, or security consequences of the above. We excluded work on training-time efficiency without an inference-memory component, and modality-specific methods (for example vision-language caches) except where they illustrate a general principle. Each method is described by a common set of axes used throughout: the underlying mechanism, the achievable memory reduction, the accuracy impact, the deployment prerequisite (training-free versus requiring calibration or pre-training), and the hardware bottleneck it addresses. These axes structure the comparative synthesis in Section 8.

Figure 2 summarises the resulting four-layer taxonomy and the cross-cutting concerns that the survey treats jointly.

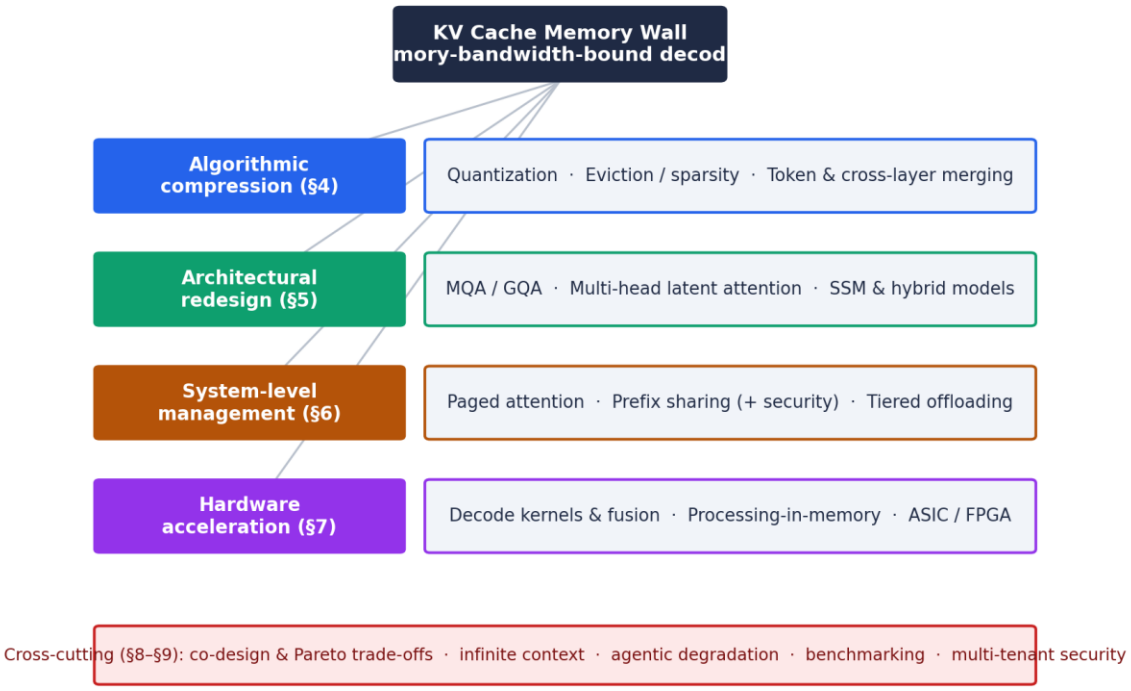

Fig. 2. Four-layer taxonomy of KV cache optimisation used in this survey, with the cross-cutting concerns of Sections 8 and 9.

3. Theoretical Foundations of the KV Cache

3.1 Self-attention and autoregressive decoding

Multi-head self-attention is the primitive underlying modern LLMs. For an input sequence of length T and hidden dimension d, each attention head forms query, key, and value projections Q, K, V in R^(T x d_k), where d_k is the per-head dimension, and computes

$$Attention(Q, K, V) = softmax(Q K^T / \sqrt{d_k}) V .$$

During prefill this is a dense matrix-matrix operation with high data reuse, and the contextual relationships among all prompt tokens are captured in one parallel pass. Generation, by contrast, is autoregressive: at step t the model forms a query q_t and attends to the new key and value (k_t, v_t) together with all prior keys K_1..K_{t-1} and values V_1..V_{t-1}. A cache-free implementation must re-derive the key and value projections of all preceding tokens at every step; performing this for each of the T generated tokens incurs O(T^2) redundant projection work over the generation, and a naive re-execution of the full self-attention at every step is costlier still, at O(T^3). Inference engines therefore cache the computed key and value tensors in high-bandwidth memory and append (k_t, v_t) at each step, so each step adds only O(t) attention work and O(1) cached vectors, replacing the redundant recomputation with linear memory accumulation.

3.2 Compute-bound prefill versus memory-bound decoding

The two regimes can be separated quantitatively using the roofline model (Williams et al. 2009), which bounds achievable performance by either peak compute or peak memory bandwidth as a function of arithmetic intensity (floating-point operations per byte moved). Prefill, dominated by matrix-matrix products with high reuse, sits on the compute-bound plateau and saturates the tensor cores of the accelerator (Dao et al. 2022). Decoding relies on matrix-vector products; every generated token requires loading the model weights and the entire growing KV cache from off-chip memory while performing comparatively little arithmetic, placing it firmly in the memory-bandwidth-bound regime (Gholami et al. 2024). This divide implies that prefill and decoding optimisations are largely distinct problems, a distinction we maintain throughout and which the roofline-based survey of Yuan et al. 2024b applies to LLM inference more broadly.

3.3 Quantitative KV cache footprint

The static cache footprint is predictable. For batch size B, total context length T, L transformer layers, H_kv key-value heads, per-head dimension D_head, and P bytes per element, the KV cache size is

$$M\_KV = 2 \cdot B \cdot T \cdot L \cdot H\_kv \cdot D\_head \cdot P \,,$$

where the factor of two accounts for separate key and value tensors. As a concrete illustration, a 70-billion-parameter model in 16-bit precision (P = 2) with context length 8192 and batch size 16 carries roughly 140 GB of static weights, while the KV cache exceeds 40 GB under a grouped-query configuration (L = 80, H_kv = 8, D_head = 128); under full multi-head attention the same setting would require several hundred gigabytes (Hooper et al. 2024a). Scaling the context length toward 10^5 tokens for retrieval-augmented generation, or increasing the batch size for throughput, drives the cache past the weight footprint, at which point it becomes the hard constraint on deployable hardware.

3.4 Memory traffic and I/O constraints

The static footprint understates the problem because most throughput degradation arises from traffic during active inference. Generating one token requires moving the entire KV cache across the high-bandwidth-memory interface (Kwon et al. 2023). With a device bandwidth of, for example, 2000 GB/s and a 40 GB cache, a single sequence is bounded near 50 tokens per second purely by cache movement, before accounting for weights or system overhead; realised throughput is lower still. Fragmentation compounds the issue: naive contiguous allocation for variable-length sequences produces non-contiguous, misaligned memory access patterns that depress effective bandwidth well below the silicon limit (Kwon et al. 2023). Addressing this requires changes at the kernel, allocator, and architecture levels, which the remaining sections survey.

4. Algorithmic Compression and Data Reduction

4.1 Low-precision quantization

Quantization reduces the number of bits used to represent each stored element. It is useful to distinguish weight quantization, which compresses the static model parameters, from KV cache quantization, which compresses the dynamic attention states and reduces the factor P in the footprint equation. The two share techniques but pose different problems, and this survey is concerned primarily with the latter; we summarise the weight-quantization heritage only where its principles transfer.

4.1.1 Activation outliers and channel-wise variance

The principal obstacle, established first in the weight- and activation-quantization literature, is the presence of outliers: a small number of channels exhibit magnitudes far larger than the rest, and uniform quantization either clips them, corrupting the representation, or widens the step size, injecting large quantization noise into the majority of well-behaved values (Liu et al. 2024f). Two principles developed for weight quantization carry over to the cache: migrating outlier magnitude from activations to weights through per-channel smoothing, as in SmoothQuant (Xiao et al. 2023); profiling the activation distribution to protect salient channels, as in AWQ (Lin et al. 2024c); and dense-and-sparse decomposition that isolates outliers in higher precision while aggressively quantizing the remainder, as in SpQR (Dettmers et al. 2023b) and SqueezeLLM (Kim et al. 2024b). In production serving, eight-bit and FP8 KV caches are now a common, near-lossless default, with four-bit and below reserved for memory-critical deployments.

4.1.2 Per-token and per-channel scaling

Keys and values exhibit different statistics, which motivates asymmetric scaling. KIVI (Liu et al. 2024f) observes that key tensors have low variance along the token axis but high variance across channels, so per-channel scales preserve their distribution, whereas value tensors vary along the token axis and are better served by per-token scales. This key-per-channel / value-per-token pattern recurs across subsequent methods, including KVQuant (Hooper et al. 2024a), WKVQuant (Yue et al. 2024), SKVQ (Duanmu et al. 2024), and the edge-oriented SubKV (Zeng et al. 2025). Applying an orthogonal rotation before quantization, which spreads outlier energy evenly across channels, is a further error-reduction technique whose use of incoherence processing originates in weight-quantization work such as QuIP (Chee et al. 2023).

4.1.3 Ultra-low precision and post-training calibration

Below four bits, quantization noise becomes strongly non-linear and base accuracy must be actively recovered. Accurate post-training quantization using Hessian-based error compensation, as in GPTQ (Frantar et al. 2023b), and incoherence-processed two-bit methods with error guarantees, as in QuIP (Chee et al. 2023), establish the feasibility of very low bit widths, while reordering and channel scaling improve low-bit serving (Zhao et al. 2024). GEAR (Kang et al. 2024) combines low-bit quantization with a low-rank term for the quantization residual and a sparse term for outliers to reach near-lossless 4-bit caches, and coupled quantization (Zhang et al. 2024c) exploits inter-channel dependence to push toward one bit per channel. At the model level, extreme weight quantization, including BitNet (Wang et al. 2023) and 1.58-bit models (Ma et al. 2024), reduces matrix multiplication to addition. For the cache specifically, two-bit and lower

quantization is feasible but its robustness is task-dependent, and aggressive settings should be validated against instruction-following and long-context behaviour rather than perplexity alone (Chen et al. 2025a).

A distinct and now-dominant line attacks outliers geometrically through rotation. Applying a computationally invariant orthogonal (often Hadamard) transform spreads outlier energy across channels so that all weights, activations, and the KV cache can be quantized to four bits without retaining any channel in higher precision, as in QuaRot (Ashkboos et al. 2024); learning the rotation rather than fixing it (Liu et al. 2024e) and combining rotation with permutation to handle massive outliers (Lin et al. 2024b) narrow the gap to full precision further. RotateKV (Su et al. 2025) specialises outlier-aware rotation to two-bit KV caches, reporting under 0.3 perplexity degradation, and mixed-precision schemes such as ResQ (Saxena et al. 2024b) keep a small high-variance low-rank subspace in higher precision while quantizing the rest. Rotation is attractive because it is calibration-light and fuses into existing projections, but it interacts with rotary position embeddings (Su et al. 2024), which several KV-specific methods must explicitly accommodate. A practical middle ground is mixed-precision allocation that assigns precision per layer or per token from attention statistics (Lei et al. 2026), keeping sensitive entries in higher precision while aggressively quantizing the rest.

4.2 Sparsification, token pruning, and eviction

4.2.1 Attention-guided eviction and heavy hitters

Attention probability mass concentrates on a small set of context tokens, so low-importance tokens can be evicted with limited quality loss. H2O (Zhang et al. 2023) maintains a fixed-budget cache by tracking accumulated attention and evicting the lowest-scoring tokens. SnapKV (Li et al. 2024c) reduces the cost of this tracking by selecting important tokens from the post-prefill attention pattern. Scissorhands (Liu et al. 2023a) formalises the persistence-of-importance hypothesis, namely that tokens important early remain important, which licenses static budget selection; TOVA (Oren et al. 2024) frames the same fixed-budget idea by viewing a decoder as a bounded multi-state RNN whose state size is the cache budget. FastGen (Ge et al. 2024) profiles each attention head and assigns it a tailored policy, retaining local context for local heads and special tokens for special-token heads, while CAKE (Qin et al. 2025b) allocates the eviction budget across layers according to their measured preferences. A complementary line exploits the observation that only certain retrieval heads need the full context: DuoAttention (Xiao et al. 2024b) keeps a full cache for retrieval heads and a streaming cache for the rest. Related work exploits contextual sparsity to skip loading large parts of the cache (Liu et al. 2023b), prunes parameters and their cached vectors in one shot (Frantar and Alistarh 2023a), and selectively fetches only high-score entries to save bandwidth (Ribar et al. 2024; Yang et al. 2024c). Natively trainable sparse-attention architectures such as NSA (Yuan et al. 2025) push this further by learning a hardware-aligned sparse pattern during pre-training rather than imposing it post hoc.

A second axis concerns how a fixed global budget is distributed rather than which tokens are scored. Uniform per-layer, per-head allocation is suboptimal because attention patterns differ across the network. Ada-KV (Feng et al. 2024) derives a head-wise adaptive allocation from a bound on the eviction-induced attention error, PyramidKV (Cai et al. 2024b) allocates more budget to lower layers following the observed pyramidal funnelling of information, and LAVa

(Shen et al. 2025) jointly sets layer and head budgets by minimising residual-stream information loss. These methods report near-lossless quality at 10-15 percent cache retention. Query-aware selection is complementary: Quest (Tang et al. 2024b) estimates page criticality from the current query and loads only the top pages, and SAGE-KV (Wang et al. 2025a) performs a one-shot token- and head-level selection after prefill. Because all of these rest on a stability-of-importance assumption that can fail, recent work explicitly bounds the worst-case risk of mis-eviction (Feng et al. 2025). Where exact recall is required, LESS (Dong et al. 2024) augments an eviction cache with a small constant-size recurrent state so that discarded tokens remain partially queryable.

A recurring refinement concerns the scoring signal itself. Pure attention-weight scores ignore the magnitude of the value vectors and the token's actual contribution to the attention output; value-aware (Guo et al. 2024) and output-error-aware (Goel et al. 2025) criteria correct this, and NACL (Chen et al. 2024) mixes proxy-token statistics with randomised eviction to reduce attention bias. Keyformer (Adnan et al. 2024) and KeyDiff (Park et al. 2025) select a compact key set, the latter from key geometry alone so it remains FlashAttention-compatible. RazorAttention (Tang et al. 2024a) keeps a full cache only for retrieval heads and a compensation token elsewhere, mirroring the head-specialisation idea of DuoAttention. Two-stage schemes such as RocketKV (Behnam et al. 2025) combine coarse permanent eviction with fine-grained top-k sparse attention to reach very high compression. These refinements consistently improve the accuracy-at-budget trade-off over heavy-hitter baselines. What is selected, then, matters as much as how much: the eviction signal governs quality, not the budget alone. Work through the first half of 2026 continues this trajectory: ReST-KV (An et al. 2026) adds layer-wise output reconstruction with spatial-temporal smoothing, CapKV (Yang et al. 2026) recasts eviction as an information-bottleneck objective that unifies prior heuristics, LookaheadKV (Ahn et al. 2026) predicts future token utility without explicit draft generation, and Crystal-KV (Wang et al. 2026c) specialises eviction to chain-of-thought reasoning by prioritising tokens that determine the final answer.

4.2.2 Rolling buffers and attention sinks

Fixed-budget pruning ultimately fails for unbounded streams. StreamingLLM (Xiao et al. 2024c) shows that retaining a few initial tokens as attention sinks, which absorb a disproportionate share of the softmax mass and stabilise the denominator, together with a rolling window of recent tokens, supports stable generation over arbitrarily long sequences under a fixed memory budget. The same principle appears architecturally as sliding-window attention in production models such as Mistral (Jiang et al. 2023a), which bounds the cache by construction. These cache-resident methods are distinct from prompt or context compression (for example LLMLingua, Jiang et al. 2023b), which shortens the input text itself; the latter is complementary and outside our scope, although Section 9.2 returns to context curation in agentic settings.

4.2.3 Semantic and chunk-level retention

Magnitude-based eviction can discard factually important but low-attention tokens. Chunk- and cluster-level methods mitigate this: ChunkKV (Liu et al. 2025b) treats semantic chunks rather than isolated tokens as the unit of compression, ClusterKV (Liu et al. 2024c) recalls tokens at the granularity of semantic clusters, and KVzip (Kim et al. 2025) scores entries by their ability to reconstruct the original context, yielding query-agnostic compression reusable across queries.

4.2.4 Block-wise eviction and aligned memory

Evicting individual tokens fragments memory and forces unaligned fetches. Block-wise schemes such as KV-Compress (Rehg 2024) and PyramidInfer (Yang et al. 2024a) operate on aligned blocks or layer-wise budgets, preserving contiguity so that compression translates into realised throughput. Dynamic sparse-attention kernels such as MInference (Jiang et al. 2024b), nearest-neighbour retrieval over cached keys (Liu et al. 2024b), and block-distributed attention (Acharya et al. 2024) similarly avoid transferring low-utility blocks. Position-persistent selection across layers, as in TidalDecode (Yang et al. 2024b), reduces the overhead of repeatedly choosing which tokens to attend to.

4.3 Structural compaction: merging, low-rank projection, and learned compression

4.3.1 Token merging and pooling

Whereas eviction discards information irreversibly, merging fuses correlated representations. Token merging (Bolya et al. 2023), originally developed for vision transformers, applies bipartite matching to combine similar tokens and has been adapted to attention caches to approximate the fixed-size state of recurrent models, compressing the spatial dimension without wholesale deletion.

4.3.2 Cross-layer sharing and depth-wise merging

Adjacent middle-to-deep layers produce highly similar key and value tensors, creating vertical redundancy. Cross-layer attention (Brandon et al. 2024) and systematic cross-layer KV sharing (Wu and Tu 2024a) let layers reuse caches from neighbours, while the layer-condensed cache (Wu and Tu 2024b) computes and stores KV for only a subset of layers. MiniCache (Liu et al. 2024a) merges KV states across depth by interpolating their directions while preserving magnitude, effectively reducing the L factor in the footprint equation.

4.3.3 Low-rank cache projection

Eviction and merging act on the token (sequence) axis; an orthogonal family compresses the hidden-dimension axis by exploiting the low-rank structure of the key and value tensors. Palu (Chang et al. 2024) decomposes the key and value projections into low-rank factors, caches the small intermediate states, and reconstructs the tensors on the fly, reporting over 90 percent cache reduction; Eigen Attention (Saxena et al. 2024a) performs attention in a low-rank subspace, and ASVD (Yuan et al. 2023) applies activation-aware SVD that absorbs outliers into the transformed weights before truncation. Because a fixed rank is suboptimal across layers, LoRC (Zhang et al. 2024b) compresses progressively with layer-wise sensitivity, MatryoshkaKV (Lin et al. 2024a) tunes trainable orthogonal projections searchable at multiple ranks, and EliteKV (Zhou et al. 2025a) jointly selects RoPE frequencies and a low-rank projection. These methods are essentially a post-hoc, training-light route to the latent compression that MLA (Section 5.2) builds in; indeed the same SVD machinery underlies head-count reduction (Yu et al. 2024). The low intrinsic dimensionality of keys is also exploited for sparse selection: Loki (Singhania et al. 2024) ranks tokens in a low-dimensional key space, and Squeezed Attention (Hooper et al. 2024b) clusters the keys of a fixed context offline and compares queries against centroids at run time. A consistent empirical caveat is that keys are more low-rank than values, so symmetric decomposition is

suboptimal, and at matched storage, quantization often dominates pure rank reduction (Salfati 2026), which motivates the low-rank-plus-quantization hybrids of Section 4.1. Recent (2026) work sharpens both the analysis and the method: KV-CoRE (Chen et al. 2026) benchmarks the data-dependent low-rank compressibility of caches across models and languages, factored keys (Yao et al. 2026) exploit that selection needs far fewer dimensions than value transfer to shrink only the key cache, and StiefAttention (Benfenati et al. 2026) learns orthonormal projections by directly minimising decoder-output reconstruction error rather than an SVD proxy.

4.3.4 Learned and trainable compression

The methods above are training-free or training-light; a further family trains the model to compress its own cache. Dynamic Memory Compression (Nawrot et al. 2024) retrofits a pre-trained model through brief continued pre-training so that each head learns its own online merge ratio, reaching 4x cache compression that outperforms up-trained GQA and eviction. Activation Beacon (Zhang et al. 2024a) adds a compression module that progressively condenses key and value activations, and KV-Distill (Chari et al. 2025) distills long caches into short, query-independent representations. For reasoning models, whose chains of thought generate very long caches, redundancy-aware compression (Cai et al. 2025) and delayed-eviction sparsification (Lancucki et al. 2025) preserve accuracy at high ratios, and task-adaptive budgets (Zhou et al. 2024a) and bounded-memory recurrent compressors (Karami et al. 2025) round out the space. The cost is a training step, which buys higher compression at fixed quality than purely post-hoc methods.

4.4 Modality-aware compression

Although this survey centres on text, the cache bottleneck is most acute in vision-language models, where each image expands into thousands of tokens, and the text methods above transfer only imperfectly because attention heads attend unevenly across modalities. VL-Cache (Tu et al. 2024) allocates a sparsity-aware, layer-adaptive budget and a modality-aware scoring policy, retaining roughly ten percent of the cache at full accuracy; MadaKV (Li et al. 2025b) senses per-head modality preference during eviction; and AirCache (Huang et al. 2025) exploits inter-modal relevance to drop redundant visual tokens, while for visual autoregressive generation ScaleKV (Li et al. 2025c) allocates cache by scale-specific attention behaviour, and for streaming video, HERMES (Zhang et al. 2026a) treats the cache as a hierarchical memory for real-time understanding. We include these as evidence that the four-layer taxonomy generalises beyond text, while noting that a full multimodal treatment is beyond our scope.

5. Architectural Paradigm Shifts

5.1 Head-sharing attention

5.1.1 Multi-query attention

Whereas algorithmic compression reduces the cost of an existing cache, architectural change bounds the cache by construction. In standard multi-head attention, each of N query heads has its own key and value projections, so a layer with 32 heads caches 32 distinct KV tensors. Multi-query attention (Shazeer 2019) requires all query heads to share a single key and value projection,

reducing the per-layer KV footprint by the head count. The cost is a measurable quality reduction: the original formulation reports faster decoding with a small degradation in output quality, and subsequent work attributes the gap to the reduced diversity of key-value subspaces, which is most visible on tasks requiring precise retrieval and reasoning (Shazeer 2019; Ainslie et al. 2023).

5.1.2 Grouped-query attention

Grouped-query attention interpolates between the two extremes by partitioning the query heads into G groups that each share one key-value projection (Ainslie et al. 2023). With 32 query heads and G = 8, the layer maintains eight key and eight value projections, a fourfold reduction relative to 32-way multi-head attention, while retaining most of its quality; larger head counts yield correspondingly larger reductions (for example 64 query heads with eight groups give an eightfold reduction, as in Llama-2-70B). Grouped-query attention is now standard in industry-scale open models such as Mistral (Jiang et al. 2023a) and Llama, and conversion recipes allow existing multi-head checkpoints to be uptrained into it cheaply (Ainslie et al. 2023).

5.2 Low-rank latent attention

5.2.1 Multi-head latent attention

Larger context windows motivated stronger compression than head sharing alone. Multi-head latent attention, introduced in DeepSeek-V2 (DeepSeek-AI 2024), down-projects the context into a low-rank latent vector that is the only quantity cached. During decoding the up-projection matrices are absorbed into the query and output projections, so the explicit key and value tensors are never reconstructed; this absorption is precisely what makes the method bandwidth-efficient, decoupling the memory cost from the compute and compressing the representation before it reaches memory. A complication is compatibility with rotary positional embeddings (Su et al. 2024), which require a dedicated pathway alongside the latent payload to preserve relative position.

5.2.2 Post-training conversion

Pre-training a model from scratch with a latent architecture is expensive, so post-training conversion methods retrofit deployed multi-head models. These decompose the projection matrices by singular value decomposition and map the dominant components onto a low-rank structure; truncation of significant singular values can degrade reasoning, which is corrected by fine-tuning the up- and down-projections on calibration data (Meng et al. 2025; Ji et al. 2025). MHA2MLA, for example, removes RoPE from low-contribution dimensions and applies a joint SVD of keys and values, recovering quality with only 0.3-0.6 percent of the original data while reducing the Llama-2-7B cache by over 90 percent.

5.3 Non-transformer and hybrid architectures

5.3.1 State-space models

State-space models depart from attention by maintaining a fixed-size recurrent hidden state updated token by token (Gu and Dao 2024). Because the state size is independent of sequence length, memory is O(1) in the context length, removing the KV cache entirely. The cost is an information bottleneck: a fixed state cannot losslessly represent arbitrarily long inputs, and

selective state mechanisms (Gu and Dao 2024; Dao and Gu 2024) and recurrent linear attention variants such as RWKV (Peng et al. 2023) and retentive networks (Sun et al. 2023) trade exact recall for bounded memory.

5.3.2 Hybrid topologies

Hybrid models combine the unbounded recall of attention with the constant memory of recurrence. Jamba (Lieber et al. 2024) interleaves Mamba and attention layers so that most of the depth runs in constant memory while a few attention layers retain exact local KV caches for precise retrieval. You-only-cache-once (Sun et al. 2024) restructures the decoder so that a single global cache serves all cross-attention layers, and Infini-attention (Munkhdalai et al. 2024) augments local attention with a compressive memory for bounded-memory infinite context.

6. System-Level Memory Management

6.1 Virtual memory and paged attention

Architectural choices set the cache size, but realised performance also depends on how that cache is allocated at run time. Legacy engines pre-allocated a contiguous block sized to the maximum sequence length, which is wasteful because output lengths vary widely; sequences that finish early leave fragmented gaps, producing internal and external fragmentation and premature out-of-memory failures even when memory remains free (Kwon et al. 2023; Yu et al. 2022).

PagedAttention, introduced with vLLM, applies operating-system virtual memory to the cache (Kwon et al. 2023). The KV cache is divided into fixed-size blocks mapped through page tables, so logically contiguous token sequences occupy physically non-contiguous blocks allocated on demand. This eliminates external fragmentation, bounds internal fragmentation to one block, and raises memory utilisation, enabling substantially larger batches on the same hardware.

Page tables also enable sharing. When concurrent requests share a system prompt or long prefix, the common prefix is computed once and its blocks are shared through a radix-tree structure, as generalised by RadixAttention in SGLang (Zheng et al. 2024a) and prefix-aware kernels such as ChunkAttention (Ye et al. 2024). Prefix reuse across turns and requests is further exploited by CachedAttention (Gao et al. 2024) and, for retrieval-augmented workloads, by KVLink (Yang et al. 2025b), which pre-computes and reuses the caches of individual documents. The same paging substrate supports concurrent low-rank adapters in S-LoRA (Sheng et al. 2024), building on memory-efficient quantized fine-tuning (Dettmers et al. 2023a), and the FlashInfer engine (Ye et al. 2025b) provides block-sparse kernels that make these layouts efficient.

Scheduling determines how the paged cache is filled over time. Iteration-level continuous batching (Yu et al. 2022) admits and retires requests at token granularity to keep the cache busy, and chunked prefill (Agrawal et al. 2024) splits long prompts into compute-balanced chunks interleaved with ongoing decodes, trading a small prefill delay for stall-free, low-jitter generation. Real-world trace studies (Wang et al. 2025b) show that cache-reuse opportunities are highly skewed and predictable per request category, which informs eviction and admission policy. Agentic and multi-turn workloads add structure that generic policies miss: KVShare (Yang et al. 2025a) reuses caches across similar (not identical) requests with selective recomputation, and

KVFlow (Pan et al. 2025) schedules prefix eviction from an agent execution graph so that caches are retained until their next predicted use.

6.2 Multi-tenant sharing and KV cache security

Prefix sharing improves efficiency but weakens tenant isolation, and a growing body of work shows that shared caches constitute an exploitable side channel. Because a cache hit on a shared prefix is faster than a miss, an adversary who probes response times can infer whether a prefix is cached and reconstruct other users' prompts token by token. PROMPTPEEK (Wu et al. 2025) demonstrates prompt reconstruction against multi-tenant serving frameworks, and timing side-channel attacks on KV and semantic caches (Song et al. 2024) and audits of deployed APIs (Gu et al. 2025) and input-stealing timing attacks (Zheng et al. 2024b) confirm the leakage is realistic across commercial providers. Direct reconstruction from the cache itself has also been shown (Luo et al. 2025). Proposed mitigations include user-boundary cache partitioning (Pang et al. 2024), selective and sensitivity-aware sharing (Chu et al. 2025), and reversible obfuscation of cache contents (Luo et al. 2025), all of which trade some sharing benefit for provable isolation. The threat surface continues to widen in 2026: reinforcement-learned attacks reconstruct prompts far more cheaply than earlier work suggested (Wang et al. 2026a), shared prefix blocks are vulnerable to Rowhammer-style bit-flips with silent, persistent corruption (Yamamoto et al. 2026), and the KV cache itself becomes a lever for latency denial-of-service that exhausts the global cache to induce head-of-line blocking (Wang et al. 2026b). Table 2 organises the reported threats and mitigations. We treat security as a first-class design axis rather than an afterthought, since it directly constrains how aggressively prefix sharing can be deployed and, as Section 9.4 notes, interacts with compression because eviction bias can itself become an observable signal.

Table 2. Reported KV cache security threats in multi-tenant serving and their mitigations (Wu et al. 2025; Song et al. 2024; Gu et al. 2025; Luo et al. 2025; Pang et al. 2024; Chu et al. 2025).

Threat	Mechanism	What leaks	Reported mitigation
Prompt reconstruction (PROMPTPEEK)	Probing shared-prefix cache hits token by token	Other users' prompts	User-boundary partitioning; selective sharing
Timing side channel	Cache-hit vs miss latency on KV / semantic cache	System and peer prompts	Constant-time / isolated serving
API-level cache audit	Statistical timing audit of public APIs	Global cache sharing, model details	Per-user caches; disclosure
Direct cache inversion	Reconstruction from cached KV contents	Input text	Reversible cache obfuscation (KV-Cloak)

6.3 Tiered offloading and latency hiding

Local high-bandwidth memory is finite, typically tens of gigabytes per accelerator, so orchestration frameworks build a tiered hierarchy. When high-bandwidth memory is exhausted, FlexGen (Sheng et al. 2023) and related systems offload cached tensors over PCIe to host DRAM and then to NVMe storage, and LLM-in-a-flash (Alizadeh et al. 2024) streams parameters from flash. KVCache-centric disaggregation, as in Mooncake (Qin et al. 2025a), and statistical

multiplexing across a cluster (Li et al. 2023b; Aminabadi et al. 2022; Holmes et al. 2024) extend effective capacity further. The penalty is retrieval latency that is orders of magnitude higher than local memory, which must be hidden. InfiniGen (Lee et al. 2024) speculates which cache entries the next layer will need and prefetches only those, and speculative decoding (Leviathan et al. 2023), in which a small draft model proposes tokens that the target model verifies in parallel, can be overlapped with asynchronous cache prefetching so that I/O and compute proceed concurrently; self-drafting variants such as Medusa (Cai et al. 2024a) and EAGLE (Li et al. 2024b) embed the draft heads in the model and change the cache-access pattern accordingly. These techniques convert a hard capacity limit into a managed latency-throughput trade-off.

6.4 Cross-request KV transport and reuse

The cache is increasingly a first-class object that outlives a single request and moves across the memory hierarchy and the network. Reusing a context's cache across requests avoids recomputing its prefill, but fetching multi-gigabyte tensors over the network can itself dominate latency. CacheGen (Liu et al. 2024d) encodes the cache into a compact, bandwidth-adaptive bitstream for fast loading; CacheBlend (Yao et al. 2024) enables reuse of non-prefix chunks, as in retrieval-augmented generation, by selectively recomputing a small subset of cross-attention tokens; and LMCache (Cheng et al. 2025) provides an open KV layer that offloads and shares caches across engines and storage tiers. System designs organise this lifecycle: elastic memory pools (Hu et al. 2024b), hierarchical context caches (Xie et al. 2025), fault-tolerant cache streaming (Strati et al. 2024), and, for multi-agent workflows, online cross-context cache communication that realigns offsets across diverging prefixes (Ye et al. 2025a). This transport layer interacts directly with compression, since a smaller cache is cheaper to store and move, and with the security concerns of Section 6.2, since a shared, transported cache widens the attack surface.

A complementary structural choice is to physically separate the two inference regimes. Because prefill is compute-bound and decoding memory-bound (Section 3.2), co-locating them causes interference; prefill-decode disaggregation assigns each phase to different devices (Zhong et al. 2024; Hu et al. 2024a), eliminating interference and letting each phase be provisioned and parallelised independently, at the cost of transferring the KV cache between phases over the interconnect. This makes the cache itself a first-class object of cluster design, and KVCache-centric disaggregation such as Mooncake organises the entire serving system around it (Qin et al. 2025a). Through 2026 the emphasis shifts to adaptivity and cost: fine-grained, SLO-aware cache reconfiguration that decides per-layer placement at runtime (Ma et al. 2026), disaggregation tuned for multi-round and agentic workloads (He et al. 2026), and energy-aware placement with dynamic voltage-frequency scaling that cuts serving energy by up to 39-48 percent (Basit et al. 2026).

7. Hardware-Accelerated Inference

7.1 Memory-aware kernels and fusion

Below the system layer, the efficiency of the attention computation itself depends on the kernel. The original FlashAttention (Dao et al. 2022) tiles attention to avoid materialising the full score matrix and minimises global memory reads, and FlashAttention-2 (Dao 2023) improves work

partitioning. These were optimised for prefill; during decoding the query length is one, which leaves most streaming multiprocessors idle. Decoding-optimised kernels such as FlashDecoding++ (Hong et al. 2024) and the customizable FlashInfer engine (Ye et al. 2025b) partition the historical sequence dimension across all multiprocessors, compute partial attention per block, and combine the partials with a numerically stable reduction, restoring high utilisation; FlashAttention-3 (Shah et al. 2024) further exploits asynchronous, low-precision tensor-core pipelines on recent hardware.

Compression methods add their own friction, because dense contiguous kernels assume uniform-precision, regularly laid-out tensors. Computing on a quantized or sparse cache requires fetching compressed data, dequantizing, and only then performing the dot product; done naively, the extra memory traffic erases the benefit of compression. Kernel fusion, using compiler toolchains or hand-written kernels, merges dequantization and any sparse gather into the same kernel as the attention product, so compressed data is fetched once and processed while resident in registers (Kwon et al. 2023; Ye et al. 2025b). Such fusion is a prerequisite for realising the theoretical gains of aggressive compression in production.

7.2 Emerging silicon: processing-in-memory and dedicated accelerators

Kernel and allocator optimisations remain bounded by the von Neumann separation of compute and memory. Processing-in-memory (PIM) and compute-in-memory designs embed attention logic within or beside the memory arrays, so that the bandwidth-heavy, low-intensity attention GEMV executes locally and only reduced results cross the interconnect. NeuPIMs (Heo et al. 2024) and IANUS (Seo et al. 2024) pair a compute-oriented NPU with GEMV-optimised PIM, ReTransformer (Yang et al. 2020) realises attention on ReRAM, AttenPIM (Chen et al. 2025b) provides dual-mode per-bank GEMV units tailored to the two attention products, PAISE (Lee et al. 2025) schedules which operations to offload to PIM, and commercial HBM-PIM and CXL-based near-memory solutions report multi-fold throughput and energy gains on transformer inference (Kim et al. 2024a); a broader treatment appears in the compute-in-memory survey of Wolters et al. 2024. Domain-specific accelerators take a complementary route, replacing dense tensor blocks with parallel sparse-lookup and pointer-chasing engines suited to eviction and merging, and reconfigurable FPGA fabrics support algorithm-hardware co-design of quantized long-context inference, as in AccLLM (Liang et al. 2025). Hardware-software co-design of this kind is widely argued to be a key long-term route past the memory wall, although its practical impact will depend on manufacturability and on integration with the software stack.

8. Comparative Synthesis and Strategic Taxonomy

8.1 A taxonomy matrix of representative methods

The diversity of methods motivates a unified comparison. Table 3 summarises the principal families along the axes defined in Section 2: representative methods, mechanism, typical memory reduction, accuracy impact, deployment prerequisite, and the hardware bottleneck primarily addressed. The figures are indicative ranges drawn from the cited works and depend strongly on model, task, and budget; they are intended for architectural selection rather than as precise benchmarks.

Table 3. Taxonomy of KV cache optimisation families. Reduction figures are indicative ranges from the cited literature.

Family	Representative methods	Mechanism	Typical reduction	Accuracy impact	Prerequisite	Bottleneck
Quantization	KIVI, KVQuant, GEAR, Coupled Quant.	Lower bit width with asymmetric scaling	2-4x (to ~8x at <4-bit)	Low (<4-bit needs calibration)	Training-free / light calibration	Capacity, bandwidth
Eviction / sparsity	H2O, SnapKV, Scissorhands, StreamingLLM	Retain heavy hitters + recent / sink tokens	4-8x and beyond	Task-dependent; hurts long-range recall	Training-free	Capacity, bandwidth
Merging / cross-layer	ToMe, MiniCache, CLA, LCKV	Fuse similar tokens or share across layers	2-5x	Low to moderate	Training-free or light tuning	Capacity (L factor)
Architecture	MQA, GQA, MLA	Share or low-rank-project KV heads	4x to >8x	Near-zero (GQA, MLA)	Pre-training or conversion	Capacity, bandwidth
Non-transformer	Mamba, RWKV, RetNet, Jamba	Fixed-size recurrent / hybrid state	O(1) memory	Recall loss; hybrids recover it	Pre-training	Capacity (eliminates cache)
System	PagedAttention, RadixAttention, FlexGen	Paging, prefix sharing, tiered offload	Higher utilisation; virtually unbounded	Lossless	Engine integration	Fragmentation, capacity
Hardware	FlashDecoding++, NeuPIMs, PIM/FPGA	Decode kernels, in-memory compute	Bandwidth / energy gains	Lossless	Specialised hardware	Bandwidth, von Neumann

To complement the qualitative families, Table 4 lists representative methods with the headline results reported by their authors. Because settings differ across papers (model, context length, budget, and task), the figures are not directly comparable and are reproduced only to indicate the order of magnitude of the achievable trade-offs; the absence of a common evaluation protocol is itself an open problem discussed in Section 9.3.

Table 4. Representative methods and author-reported results. Settings differ and the figures are not directly comparable (see Section 9.3).

Method	Type	Reported setting	Reported result
MiniCache	Cross-layer merge	LLaMA-2-7B, 4-bit, ShareGPT	up to 5.0x compression, ~5x throughput, near-lossless
GEAR	Quant + low-rank + sparse	4-bit KV	2.29x peak-memory and 2.38x throughput, near-lossless

PyramidInfer	Layer-wise eviction	throughput benchmark	>54% KV memory cut, 2.2x throughput
ClusterKV	Semantic-cluster recall	32K context, 1-2K budget	~2x faster latency, ~2.5x throughput, negligible loss
KVzip	Context-reconstruction eviction	up to 170K context	3-4x cache cut, ~2x decode speedup, negligible loss
Coupled Quant.	Inter-channel quantization	low-bit KV	model quality preserved down to ~1 bit/channel
InfiniGen	Speculative offload prefetch	offloading-based serving	up to 3.0x over prior KV management
PyramidKV	Layer-adaptive budget	LongBench	full-cache quality at ~12% retention
Quest	Query-aware page selection	long-dependency tasks	up to 2.23x self-attention speedup, negligible loss
QuaRot	Rotation-based 4-bit	LLaMA-2-70B, W4A4KV4	<=0.47 perplexity loss, ~99% zero-shot retained

Table 5 collates a frequently reported operating point, accuracy retention at a stated KV budget on LongBench-style long-context tasks, across eviction, merging, low-rank, and learned methods. It is assembled from the cited papers and remains subject to the comparability caveat of Section 9.3 (models and task subsets differ); it is included because the budget-versus-retention framing is the axis the field implicitly converges on, and it is precisely this axis that the MBE protocol of Section 10 standardises so that such a table can be produced under controlled conditions.

Table 5. Author-reported accuracy retention at a stated KV budget, collated from the cited works. Settings differ; not a controlled comparison (cf. the MBE protocol, Section 10).

Method	Family	Model / setting	KV budget	Reported retention
H2O	heavy-hitter eviction	OPT / LLaMA, long-context	~20%	near-full on summarisation
PyramidKV	layer-adaptive eviction	LLaMA-3, LongBench	12%	matches full cache
CAKE	cascading eviction	LongBench / NeedleBench	3.2%	maintains performance
DynamicKV	task-adaptive eviction	Mistral-7B, LongBench	1.7%	~85% of full (NIAH at 0.9%)
ChunkKV	semantic-chunk	LongBench	matched ratio	+8.7% over prior SOTA
RocketKV	two-stage eviction	long-context, A100	up to ~0.25%	negligible loss (to ~400x)
Palu	low-rank projection	LLaMA, perplexity	>90% reduced	lower PPL than quant-only
DMC	learned (retrofit)	LLaMA-2 7-70B, H100	25% (4x)	preserves downstream, >GQA/H2O

Activation Beacon	learned compression	128K, NIAH / doc	12.5% (8x)	comparable to uncompressed
R-KV	learned (reasoning)	math reasoning	10-16%	~100-105% of full

The pace and composition of this literature are themselves informative. Figure 3 plots the surveyed references by year and family: activity rises sharply through 2024 and remains high into 2025-2026, and the security and agentic categories appear only in the most recent years, consistent with their status as emerging, under-surveyed concerns that this survey foregrounds.

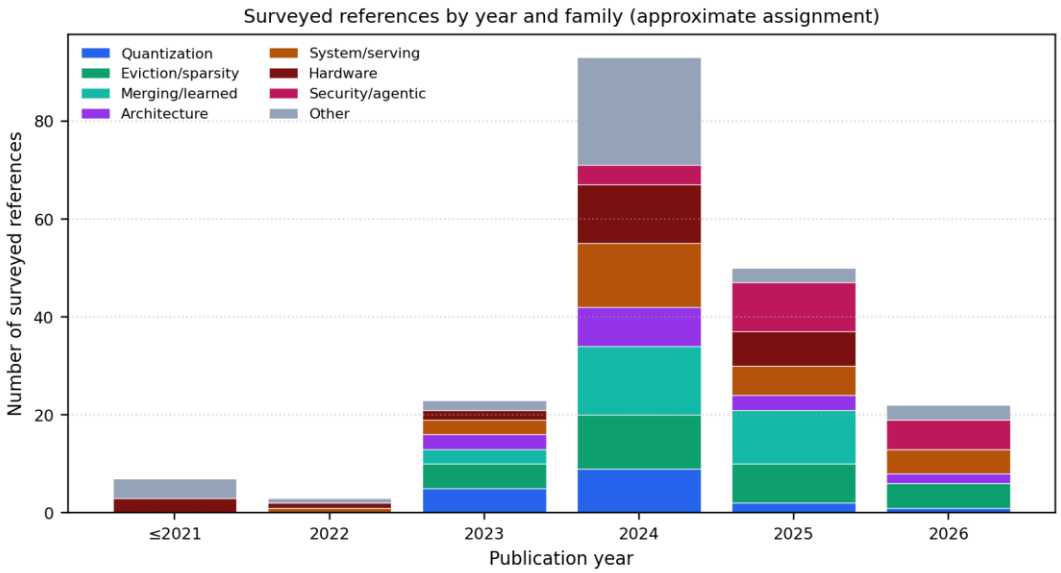

Fig. 3. Surveyed references by publication year and method family (approximate single-family assignment from the reference corpus).

8.2 Co-design and algorithmic-mathematical interference

Current practice increasingly combines orthogonal techniques, but naive stacking can be counterproductive. Eviction and quantization interfere: pruning old tokens shifts the attention distribution and creates new activation outliers that exceed the ranges assumed by a fixed quantization grid, degrading reasoning. Q-Hitter (Zhang et al. 2024f) characterises this interaction and co-designs a token oracle that is robust to quantization, demonstrating that the scales and the pruning schedule must be solved jointly rather than independently. GEAR (Kang et al. 2024) similarly composes quantization, low-rank, and sparse terms so that their errors are complementary. As a rough guide, quantization composes well with paging, offloading, and head-sharing architectures because these are largely orthogonal to numerical precision; quantization composed with aggressive eviction requires joint design to avoid outlier interference; and two lossy compressors of the same kind (for example eviction plus merging) tend to compound errors unless one is made aware of the other. The general principle is that memory reductions multiply only when the methods are mutually aware.

8.3 Performance-accuracy Pareto frontiers

Deployment decisions are best framed as a three-dimensional Pareto trade-off among memory reduction, accuracy preservation, and the latency overhead the compression itself introduces, illustrated in Figure 4 using the author-reported points of Table 5. The accuracy axis is strongly task-dependent, but not in the direction sometimes assumed: multi-step reasoning, such as chain-of-thought arithmetic on GSM8K, is among the most sensitive workloads, because every intermediate step must remain retrievable, and multi-document tasks such as those in LongBench (Bai et al. 2024) and multi-instruction prompts (Chen et al. 2025a) degrade sharply once distant evidence is destroyed; simpler local tasks tolerate more aggressive budgets. The latency axis matters because some methods, for example bipartite token merging, trade a bandwidth bottleneck for a compute-bound one through expensive matching during decoding. The preferred operating point reduces high-bandwidth-memory pressure without adding per-token latency or sacrificing downstream task accuracy, and it must be configured per workload rather than chosen once globally.

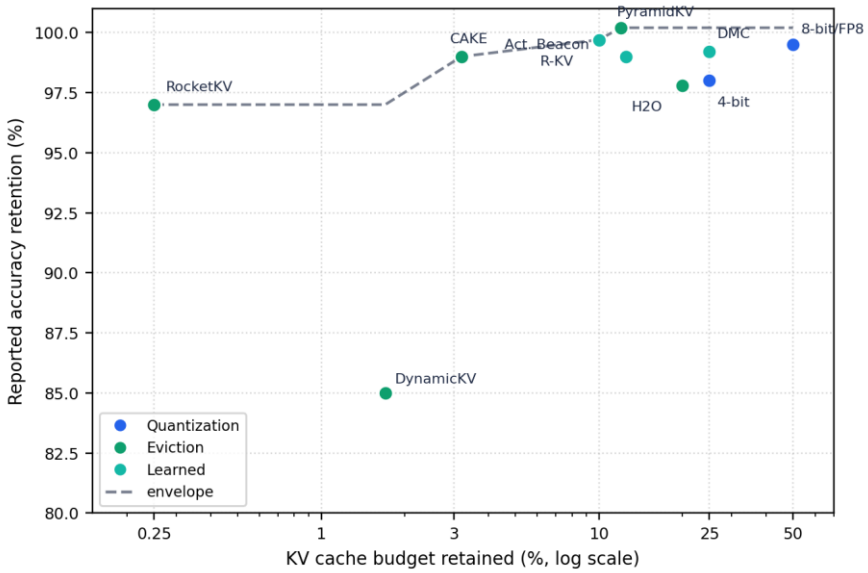

Fig. 4. Memory-accuracy frontier plotted from author-reported points (Table 5). Settings differ across papers, so positions are indicative, not a controlled comparison; the MBE harness (Section 10) produces the controlled version.

9. Open Challenges and Future Directions

9.1 Infinite-context serving

Architectural proposals now advertise context windows of a million tokens or more, but caching a million-token sequence in 16-bit precision can require hundreds of gigabytes even under grouped-query attention, exceeding a single accelerator and straining a multi-accelerator node, so the advertised window is rarely the economically deployable one. Rolling-buffer methods such as StreamingLLM (Xiao et al. 2024c) and compressive memories such as Infini-attention (Munkhdalai et al. 2024) keep memory bounded, and positional-extrapolation methods such as

YaRN (Peng et al. 2024), LongRoPE (Ding et al. 2024), LM-Infinite (Han et al. 2024), and the training-free InfLLM (Xiao et al. 2024a) extend the usable window. The unresolved problem is that fixed-size buffers forget everything beyond the window and hallucinate when queried about it; bounding memory while retaining access to arbitrarily distant facts, without re-accumulating tensors, remains open.

9.2 Degradation in long-horizon agentic loops

Aggressive eviction fails more severely in agentic settings, where models recursively read tool outputs back into their own context. Such traces exhibit two properties that ordinary eviction heuristics handle poorly: the cache grows rapidly and unpredictably as external content is appended, and operationally critical tokens, such as a base instruction issued at the start of the trace, carry low running attention yet must never be discarded. At least three failure modes follow. First, instruction loss: as attention to an early constraint fades over thousands of tool calls, a naive manager evicts it and the agent violates its own rules. Second, evidence loss: distant retrieved facts needed for a late sub-goal are pruned before use. Third, distraction: unbounded accumulation of low-value observations dilutes attention and degrades reasoning even when nothing is evicted. SideQuest (Kariyappa et al. 2026) addresses the first two by reframing cache management as a model-driven auxiliary task in which the reasoning model itself judges token usefulness, and learned context-curation methods such as MEM1 (Zhou et al. 2025b), ACON (Kang et al. 2025), and memory-as-action (Zhang et al. 2025) treat retention as a learned policy rather than a fixed heuristic; reinforcement-learned memory managers such as Memory-R1 (Yan et al. 2025) learn explicit add, update, and delete operations, and the broader design space is surveyed by Zhang et al. 2024e. Evaluation is itself immature: dedicated benchmarks (Hu et al. 2025; Zhao et al. 2026) show that current memory systems master none of accurate retrieval, test-time learning, long-range understanding, and selective forgetting simultaneously. Causal- and experience-graph memories (Zhang et al. 2026b; Ye et al. 2026) are the 2026 response, but they remain decoupled from the hardware-level cache they ultimately consume. These policies are typically evaluated on task accuracy rather than on their interaction with hardware-efficient, contiguous cache layouts, and reconciling learned retention with paged, block-aligned memory remains an open systems problem at the intersection of this survey's four layers.

9.3 The benchmarking deficit

Progress is hard to measure because standard benchmarks do not test what compression breaks. Multiple-choice and perplexity benchmarks probe knowledge stored in weights, not operation under a constrained KV memory during generation. Chen et al. 2025a show empirically that compressed models can retain perplexity and pass standard benchmarks while failing multi-instruction following, and that specific instructions degrade far faster than others, with system-prompt leakage as a concrete failure. Needle-in-a-haystack retrieval is now common but superficial; RULER (Hsieh et al. 2024), HELMET (Yen et al. 2025), L-Eval (An et al. 2024), LV-Eval (Yuan et al. 2024a), LooGLE (Li et al. 2023a), the 100K-plus Infinity-Bench (Zhang et al. 2024d), and NoLiMa (Modarressi et al. 2025) demonstrate that models passing simple retrieval still fail multi-hop tracing, aggregation, and non-literal matching as length grows. KV-cache-centric analyses such as SCBench (Li et al. 2025d) begin to close this gap on the task side. What remains missing is a reporting convention that makes results comparable: methods are

evaluated at different budgets, on different models, with different system metrics. We address this directly in Section 10 with the Matched-Budget Evaluation protocol, which standardises what to report and at which fixed budgets, and which would make the author-reported figures of Table 4 directly comparable.

9.4 Multi-tenant isolation and cache security

Section 6.2 and Table 2 catalogue the demonstrated attacks; the open problem is a defence that does not surrender the efficiency that motivates sharing. Current mitigations sit at the extremes: full per-tenant isolation removes the leak but forfeits prefix reuse, while selective and sensitivity-aware schemes recover most of the benefit only under assumptions about which prefixes are sensitive. A principled, low-overhead isolation model with provable guarantees, integrated into the paging and scheduling layers rather than bolted on afterwards, is needed. The problem is sharpened by its interaction with compression: an eviction policy whose decisions depend on content can itself become an observable signal, so security and compression must be co-designed rather than analysed in isolation.

10. Matched-Budget Evaluation: A Standardised Reporting Protocol

Section 9.3 argued that the central obstacle to measurable progress is not a shortage of methods but the absence of a common reporting convention: published results use different models, budgets, tasks, and system metrics, so the headline numbers of Table 4 cannot be placed on a single axis. This section specifies Matched-Budget Evaluation (MBE), a lightweight protocol that prescribes what a KV cache compression method should report and under which fixed conditions. MBE is deliberately not a new benchmark; it is a reporting layer that consumes existing task suites such as LongBench (Bai et al. 2024), RULER (Hsieh et al. 2024), and the KV-cache-centric SCBench (Li et al. 2025d), and fixes the axes along which their results are compared.

10.1 The matched-budget principle

The defining choice of MBE is to compare methods at fixed key-value memory budgets rather than at each method's own preferred operating point. A budget is expressed as the fraction of the full-cache footprint retained, computed from the footprint equation of Section 3.3, and methods are required to report results at a common ladder of budgets: 50, 25, and 12.5 percent, with an optional aggressive 6.25 percent point. Fixing the budget removes the most common source of incomparability, in which one method reports near-lossless quality at a generous budget while another reports a higher compression factor at lower quality, and the two cannot be ranked. Under MBE, every method is read at the same memory cost, so the accuracy-at-budget curve, rather than a single self-selected number, becomes the unit of comparison.

10.2 The reporting suite

MBE fixes three axes. The model axis specifies a small, openly available suite spanning scales and attention designs, so that results are not tied to a single architecture. The task axis spans the behaviours that Section 9.3 showed degrade differently under compression: long-document retrieval and question answering, multi-hop tracing and aggregation, instruction following under multiple instructions, chain-of-thought reasoning (which Section 8.3 identified as compression-

sensitive), and at least one multi-turn or agentic trace to exercise the failure modes of Section 9.2. The system axis requires the operational quantities that accuracy-only reporting omits: peak KV memory, decode throughput, time-to-first-token, the largest batch served before out-of-memory, and the declared hardware tier. Table 6 summarises the specification.

Table 6. The Matched-Budget Evaluation (MBE) reporting specification. Methods report the task-accuracy grid and system metrics at each fixed budget.

Axis	Required dimension	Specified values / what to report
Budget	Retained KV fraction	50% / 25% / 12.5% (optional 6.25%), computed from the Section 3.3 footprint
Model	Scale and attention type	A small open suite covering a 7-8B GQA model, a second 7-14B model, and one >=70B model
Task	Retrieval	Long-document QA (e.g., LongBench, SCBench tasks)
Task	Aggregation / tracing	Multi-hop and aggregation (e.g., RULER)
Task	Instruction following	Multi-instruction prompts (sensitivity per Chen et al. 2025a)
Task	Reasoning	Chain-of-thought arithmetic (compression-sensitive)
Task	Agentic / multi-turn	At least one long-horizon trace (per Section 9.2)
System	Memory and latency	Peak KV memory, decode throughput, TTFT, max batch before OOM, hardware tier
Method	Deployment prerequisite	Training-free vs calibration vs pre-training; compatibility (see Section 8.2)

10.3 The KV compression card

To make compliance concrete and citable, MBE packages a method's results as a single standardised KV Compression Card, analogous in spirit to model cards for model reporting (Mitchell et al. 2019). A card records, for one method on one model: the accuracy-at-budget grid across the task suite, the system metrics at each budget, the deployment prerequisite, and a compatibility row that states which other technique families the method composes with, conflicts with, or requires joint design with, drawing on the interference analysis of Section 8.2. The card is the unit that authors fill in and that the survey's accompanying repository aggregates into a public leaderboard, so that a method's standing is a matter of a submitted, reproducible artifact rather than a self-reported headline.

10.4 The open harness and reference results

To lower the barrier to adoption, the protocol is released as an open, configuration-driven evaluation harness with method adapters for several widely used open-source techniques (covering quantization, heavy-hitter eviction, sliding-window, and layer-adaptive methods), together with the KV Compression Card template and a versioned leaderboard. A seed set of reference cards, produced by this harness on the 7-8B models of the suite under the fixed budget ladder, will populate the leaderboard and provide the first internally comparable cross-method picture, replacing the cross-setting caveats that necessarily attach to the author-reported figures of

Tables 4 and 5; at the time of writing the harness and templates are released and the seed run is in progress, with cells reported only once measured. As an end-to-end validation, a small CPU proof-of-concept on a 0.5-billion-parameter grouped-query model reproduces the expected behaviour of the quantization family under matched budgets: eight- and four-bit key-value caches are lossless on single-needle retrieval, whereas two-bit caches collapse, consistent with the fragility discussed in Section 4.1.3; this is a functionality check, not the model-scale result, which the seed run on the full suite will provide. External methods are added by submitting a card, so the comparison stays current as the field moves. We position MBE not as a final answer but as a minimal, adoptable convention whose value grows with participation, and whose axes (model suite, task suite, budget ladder) are intended to be revised in the open as new task and system concerns emerge.

10.5 Limitations and threats to validity

We state the limits of this work explicitly. First, the survey is narrative and structured rather than exhaustive: although the protocol of Section 2 is reproducible, the field publishes faster than any fixed list can track, and methods appearing after the June 2026 cut-off are necessarily absent. Second, the quantitative figures collated in Tables 4 and 5 are author-reported under heterogeneous settings and are not directly comparable; we present them as order-of-magnitude evidence, and indeed the central purpose of MBE (Section 10) is to remove this incomparability. Third, MBE is introduced as a specification with an open harness and seed reference cards; broad community results under the protocol will accrue only with adoption, and its axes (model suite, task suite, budget ladder) are deliberately revisable in the open rather than fixed by fiat. Fourth, our deepest treatment is of text-only decoder LLMs; multimodal and encoder-decoder caches are surveyed only briefly (Section 4.4). We regard these as scoping choices rather than oversights, and flag them so that readers can calibrate the survey's claims.

11. Conclusion

The trajectory of large language model deployment is bound by the memory architecture of the underlying hardware. We have argued against the view that scaling inference is purely a compute problem: while prefill is compute-bound, autoregressive decoding is governed by the memory wall, and the KV cache is the object that turns generation from a compute-bound into a bandwidth-bound task.

We surveyed four complementary layers of mitigation. Algorithmic methods - low-precision quantization, attention-guided eviction, and structural merging - reduce the stored or moved bytes within the existing architecture. Architectural methods - grouped-query and latent attention, and recurrent or hybrid state-space models - bound the cache by construction. System methods - paged virtual memory, prefix sharing, and tiered offloading - raise utilisation and extend effective capacity. Hardware methods - decoding-optimised kernels, fusion, and processing-in-memory - attack the bandwidth limit at the silicon level. These are complementary to compute-reduction techniques such as sparsely activated mixture-of-experts models (Jiang et al. 2024a), which lower active computation but not the KV cache and therefore leave the memory wall in place. No single technique dominates: the right choice depends on whether the deployment prioritises exact long-

range recall, edge memory limits, or the freedom to pre-train, and the largest gains come from co-designing methods so their errors remain complementary.

Two concerns deserve sustained attention as the field advances. Multi-tenant KV cache sharing introduces real and demonstrated security risks that constrain how aggressively efficiency optimisations can be deployed, and the absence of standardised, memory-specific, end-to-end evaluation obscures genuine progress and the degradation that compression introduces in long-context and agentic settings. To make that progress measurable, we proposed Matched-Budget Evaluation (Section 10), a reporting protocol and open harness under which methods are compared at fixed memory budgets; we hope it lowers the barrier to honest, comparable evaluation. Convergence of integrated compression, principled software memory management, and purpose-built hardware offers a credible path past the memory wall, provided that evaluation and isolation keep pace with efficiency.

Declarations

Funding. The author declares that no funds, grants, or other support were received during the preparation of this manuscript.

Competing interests. The author has no relevant financial or non-financial interests to disclose.

Data availability. No new datasets were generated. The Matched-Budget Evaluation (MBE) protocol specification, the open evaluation harness, the KV Compression Card template, and the seed reference results described in Section 10 are released in a public repository at https://github.com/rohithreddybc/mbe-protocol; all other works discussed are cited and publicly available.

Ethics approval. Not applicable. This review does not involve human participants, their data, or animals.

Author contributions. R.R. is the sole author and is responsible for the conception, literature analysis, software (the MBE harness), visualisation, and writing of the manuscript (CRediT: Conceptualization, Investigation, Software, Visualization, Writing - original draft, Writing - review & editing).

Use of AI tools. Any use of generative AI tools was limited to language editing and reference organisation; all technical content, analysis, and conclusions are the authors' own and were verified against primary sources.

References

Acharya, S., et al. (2024). Star Attention: efficient LLM inference over long sequences. arXiv preprint arXiv:2411.17116.

Adnan, M., et al. (2024). Keyformer: KV cache reduction through key tokens selection for efficient generative inference. In Proceedings of Machine Learning and Systems (MLSys) 6.

Agrawal, A., et al. (2024). Taming throughput-latency tradeoff in LLM inference with Sarathi-Serve. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI) (pp. 117-134).

Ahn, J., et al. (2026). LookaheadKV: fast and accurate KV cache eviction by glimpsing into the future without generation. In International Conference on Learning Representations (ICLR 2026); arXiv:2603.10899.

Ainslie, J., Lee-Thorp, J., de Jong, M., Zemlyanskiy, Y., Lebron, F., & Sanghai, S. (2023). GQA: training generalized multi-query transformer models from multi-head checkpoints. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP) (pp. 4895-4901).

Alizadeh, K., et al. (2024). LLM in a flash: efficient large language model inference with limited memory. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL) (pp. 12562-12584).

Aminabadi, R. Y., et al. (2022). DeepSpeed-Inference: enabling efficient inference of transformer models at unprecedented scale. In Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC).

An, C., et al. (2024). L-Eval: instituting standardized evaluation for long context language models. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL) (pp. 14388-14411).

An, Y., et al. (2026). ReST-KV: robust KV cache eviction with layer-wise output reconstruction and spatial-temporal smoothing. OpenReview (forum PhEHuo7oMm).

Ashkboos, S., et al. (2024). QuaRot: outlier-free 4-bit inference in rotated LLMs. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Bai, Y., et al. (2024). LongBench: a bilingual, multitask benchmark for long context understanding. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL) (pp. 3119-3137).

Basit, O., et al. (2026). DualScale: energy-efficient disaggregated LLM serving via phase-aware placement and DVFS. arXiv preprint.

Behnam, P., et al. (2025). RocketKV: accelerating long-context LLM inference via two-stage KV cache compression. arXiv preprint arXiv:2502.14051.

Benfenati, L., et al. (2026). Don't be so Stief! Learning KV cache low-rank approximation over the Stiefel manifold. arXiv preprint arXiv:2601.21686.

Bolya, D., Fu, C.-Y., Dai, X., Zhang, P., Feichtenhofer, C., & Hoffman, J. (2023). Token merging: your ViT but faster. In International Conference on Learning Representations (ICLR).

Brandon, W., Mishra, M., Nrusimha, A., Panda, R., & Ragan-Kelley, J. (2024). Reducing transformer key-value cache size with cross-layer attention. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Cai, T., et al. (2024a). Medusa: simple LLM inference acceleration framework with multiple decoding heads. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Cai, Z., et al. (2024b). PyramidKV: dynamic KV cache compression based on pyramidal information funneling. arXiv preprint arXiv:2406.02069.

Cai, Z., et al. (2025). R-KV: redundancy-aware KV cache compression for training-free reasoning models acceleration. arXiv preprint arXiv:2505.24133.

Chang, C.-C., et al. (2024). Palu: KV-cache compression with low-rank projection. arXiv preprint arXiv:2407.21118.

Chari, V., et al. (2025). KV-Distill: nearly lossless learnable context compression for LLMs. arXiv preprint arXiv:2503.10337.

Chee, J., Cai, Y., Kuleshov, V., & De Sa, C. (2023). QuIP: 2-bit quantization of large language models with guarantees. In Advances in Neural Information Processing Systems (NeurIPS) 36.

Chen, A., et al. (2025a). The pitfalls of KV cache compression. arXiv preprint arXiv:2510.00231.

Chen, J., et al. (2026). KV-CoRE: benchmarking data-dependent low-rank compressibility of KV-caches in LLMs. arXiv preprint arXiv:2602.05929.

Chen, L., et al. (2025b). AttenPIM: accelerating LLM attention with dual-mode GEMV in processing-in-memory. In Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC).

Chen, Y., et al. (2024). NACL: a general and effective KV cache eviction framework for LLM at inference time. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL).

Cheng, Y., et al. (2025). LMCache: an efficient KV cache layer for enterprise-scale LLM inference. arXiv preprint arXiv:2508.18203.

Chu, K., et al. (2025). Selective KV-cache sharing to mitigate timing side-channels in LLM inference (SafeKV). arXiv preprint arXiv:2508.08438.

Dao, T. (2023). FlashAttention-2: faster attention with better parallelism and work partitioning. arXiv preprint arXiv:2307.08691 (presented at ICLR 2024).

Dao, T., & Gu, A. (2024). Transformers are SSMs: generalized models and efficient algorithms through structured state space duality. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., & Re, C. (2022). FlashAttention: fast and memory-efficient exact attention with IO-awareness. In Advances in Neural Information Processing Systems (NeurIPS) 35 (pp. 16344-16359).

DeepSeek-AI. (2024). DeepSeek-V2: a strong, economical, and efficient mixture-of-experts language model. arXiv preprint arXiv:2405.04434.

Dettmers, T., et al. (2023b). SpQR: a sparse-quantized representation for near-lossless LLM weight compression. In International Conference on Learning Representations (ICLR 2024); arXiv:2306.03078.

Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023a). QLoRA: efficient finetuning of quantized LLMs. In Advances in Neural Information Processing Systems (NeurIPS) 36.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: pre-training of deep bidirectional transformers for language understanding. In Proceedings of NAACL-HLT (pp. 4171-4186).

Ding, Y., et al. (2024). LongRoPE: extending LLM context window beyond 2 million tokens. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Dong, H., et al. (2024). Get more with LESS: synthesizing recurrence with KV cache compression for efficient LLM inference. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Duanmu, H., Yuan, Z., Li, X., Duan, J., Zhang, X., & Lin, D. (2024). SKVQ: sliding-window key and value cache quantization for large language models. In First Conference on Language Modeling (COLM).

Feng, Y., et al. (2024). Ada-KV: optimizing KV cache eviction by adaptive budget allocation for efficient LLM inference. arXiv preprint arXiv:2407.11550.

Feng, Y., et al. (2025). Taming the fragility of KV cache eviction in LLM inference (DefensiveKV). arXiv preprint arXiv:2510.13334.

Frantar, E., & Alistarh, D. (2023a). SparseGPT: massive language models can be accurately pruned in one-shot. In Proceedings of the 40th International Conference on Machine Learning (ICML) (pp. 10323-10337).

Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2023b). GPTQ: accurate post-training quantization for generative pre-trained transformers. In International Conference on Learning Representations (ICLR).

Gao, B., et al. (2024). Cost-efficient large language model serving for multi-turn conversations with CachedAttention. In Proceedings of the 2024 USENIX Annual Technical Conference (ATC) (pp. 111-126).

Ge, S., Zhang, Y., Liu, L., Zhang, M., Han, J., & Gao, J. (2024). Model tells you what to discard: adaptive KV cache compression for LLMs. In International Conference on Learning Representations (ICLR).

Gholami, A., Yao, Z., Kim, S., Hooper, C., Mahoney, M. W., & Keutzer, K. (2024). AI and memory wall. IEEE Micro, 44(3), 33-39.

Goel, R., et al. (2025). CAOTE: KV cache selection for LLMs via attention output error-based token eviction. arXiv preprint arXiv:2504.14051.

Gu, A., & Dao, T. (2024). Mamba: linear-time sequence modeling with selective state spaces. In First Conference on Language Modeling (COLM).

Gu, C., Li, X. L., Kuditipudi, R., Liang, P., & Hashimoto, T. (2025). Auditing prompt caching in language model APIs. arXiv preprint arXiv:2502.07776.

Guo, Z., et al. (2024). Attention score is not all you need for token importance indicator in KV cache reduction: value also matters. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP).

Han, C., et al. (2024). LM-Infinite: zero-shot extreme length generalization for large language models. In Proceedings of NAACL-HLT (pp. 3991-4008).

He, W., et al. (2026). Efficient multi-round LLM inference over disaggregated serving (AMPD). arXiv preprint arXiv:2602.14516.

Heo, G., et al. (2024). NeuPIMs: NPU-PIM heterogeneous acceleration for batched LLM inferencing. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 3 (pp. 722-737).

Holmes, C., et al. (2024). DeepSpeed-FastGen: high-throughput text generation for LLMs via MII and DeepSpeed-Inference. arXiv preprint arXiv:2401.08671.

Hong, K., et al. (2024). FlashDecoding++: faster large language model inference on GPUs. In Proceedings of Machine Learning and Systems (MLSys) 6.

Hooper, C., et al. (2024a). KVQuant: towards 10 million context length LLM inference with KV cache quantization. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Hooper, C., et al. (2024b). Squeezed attention: accelerating long context length LLM inference. arXiv preprint arXiv:2411.09688.

Hsieh, C.-P., et al. (2024). RULER: what's the real context size of your long-context language models? In First Conference on Language Modeling (COLM).

Hu, C., et al. (2024a). Inference without interference: disaggregate LLM inference for mixed downstream workloads (TetriInfer). arXiv preprint arXiv:2401.11181.

Hu, C., et al. (2024b). MemServe: context caching for disaggregated LLM serving with elastic memory pool. arXiv preprint arXiv:2406.17565.

Hu, Y., et al. (2025). Evaluating memory in LLM agents via incremental multi-turn interactions (MemoryAgentBench). arXiv preprint arXiv:2507.05257.

Huang, K., et al. (2025). AirCache: activating inter-modal relevancy KV cache compression for efficient large vision-language model inference. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV).

Ji, T., et al. (2025). Towards economical inference: enabling DeepSeek's multi-head latent attention in any transformer-based LLMs (MHA2MLA). arXiv preprint arXiv:2502.14837.

Jiang, A. Q., et al. (2023a). Mistral 7B. arXiv preprint arXiv:2310.06825.

Jiang, A. Q., et al. (2024a). Mixtral of experts. arXiv preprint arXiv:2401.04088.

Jiang, H., et al. (2024b). MInference 1.0: accelerating pre-filling for long-context LLMs via dynamic sparse attention. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Jiang, H., Wu, Q., Lin, C.-Y., Yang, Y., & Qiu, L. (2023b). LLMLingua: compressing prompts for accelerated inference of large language models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP) (pp. 13358-13376).

Jiang, J., et al. (2026). Towards efficient large language model serving: a survey on system-aware KV cache optimization. In Findings of the Association for Computational Linguistics: ACL 2026.

Kang, H., et al. (2024). GEAR: an efficient KV cache compression recipe for near-lossless generative inference of LLM. arXiv preprint arXiv:2403.05527.

Kang, M., et al. (2025). ACON: optimizing context compression for long-horizon LLM agents. arXiv preprint arXiv:2510.00615.

Karami, M., et al. (2025). Trellis: learning to compress key-value memory in attention models. arXiv preprint arXiv:2510.07051.

Kariyappa, S., et al. (2026). SideQuest: model-driven KV cache management for long-horizon agentic reasoning. arXiv preprint arXiv:2602.22603.

Kim, B., et al. (2024a). The breakthrough memory solutions for improved performance on LLM inference. IEEE Micro, 44(3), 40-48.

Kim, J.-H., et al. (2025). KVzip: query-agnostic KV cache compression with context reconstruction. arXiv preprint arXiv:2505.23416.

Kim, S., et al. (2024b). SqueezeLLM: dense-and-sparse quantization. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Kwon, W., et al. (2023). Efficient memory management for large language model serving with PagedAttention. In Proceedings of the 29th Symposium on Operating Systems Principles (SOSP) (pp. 611-626).

Lancucki, A., et al. (2025). Inference-time hyper-scaling with KV cache compression. arXiv preprint arXiv:2506.05345.

Lee, H., et al. (2025). PAISE: PIM-accelerated inference scheduling engine for transformer-based LLM. In 2025 IEEE International Symposium on High-Performance Computer Architecture (HPCA).

Lee, W., Lee, J., Seo, J., & Sim, J. (2024). InfiniGen: efficient generative inference of large language models with dynamic KV cache management. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI) (pp. 155-172).

Lei, J., et al. (2026). ARKV: adaptive and resource-efficient KV cache management under limited memory budget for long-context inference in LLMs. arXiv preprint arXiv:2603.08727.

Leviathan, Y., Kalman, M., & Matias, Y. (2023). Fast inference from transformers via speculative decoding. In Proceedings of the 40th International Conference on Machine Learning (ICML) (pp. 19274-19286).

Li, H., et al. (2025a). A survey on large language model acceleration based on KV cache management. Transactions on Machine Learning Research (TMLR).

Li, J., et al. (2024a). Large language model inference acceleration: a comprehensive hardware perspective. arXiv preprint arXiv:2410.04466.

Li, J., Wang, M., Zheng, Z., & Zhang, M. (2023a). LooGLE: can long-context language models understand long contexts? arXiv preprint arXiv:2311.04939.

Li, K., et al. (2025b). MadaKV: adaptive modality-perception KV cache eviction for efficient multimodal long-context inference. arXiv preprint arXiv:2506.15724.

Li, K., et al. (2025c). Memory-efficient visual autoregressive modeling with scale-aware KV cache compression (ScaleKV). arXiv preprint arXiv:2505.19602.

Li, Y., et al. (2024b). EAGLE: speculative sampling requires rethinking feature uncertainty. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Li, Y., et al. (2024c). SnapKV: LLM knows what you are looking for before generation. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Li, Y., et al. (2025d). SCBench: a KV cache-centric analysis of long-context methods. In International Conference on Learning Representations (ICLR).

Li, Z., et al. (2023b). AlpaServe: statistical multiplexing with model parallelism for deep learning serving. In Proceedings of the 17th USENIX Symposium on Operating Systems Design and Implementation (OSDI) (pp. 663-679).

Liang, Y., et al. (2025). AccLLM: accelerating long-context LLM inference via algorithm-hardware co-design. IEEE Transactions on Very Large Scale Integration (VLSI) Systems.

Lieber, O., et al. (2024). Jamba: a hybrid transformer-Mamba language model. arXiv preprint arXiv:2403.19887.

Lin, B., et al. (2024a). MatryoshkaKV: adaptive KV compression via trainable orthogonal projection. arXiv preprint arXiv:2410.14731.

Lin, H., et al. (2024b). DuQuant: distributing outliers via dual transformation makes stronger quantized LLMs. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Lin, J., et al. (2024c). AWQ: activation-aware weight quantization for on-device LLM compression and acceleration. In Proceedings of Machine Learning and Systems (MLSys) 6.

Liu, A., Liu, J., Pan, Z., He, Y., Haffari, G., & Zhuang, B. (2024a). MiniCache: KV cache compression in depth dimension for large language models. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Liu, D., et al. (2024b). RetrievalAttention: accelerating long-context LLM inference via vector retrieval. arXiv preprint arXiv:2409.10516.

Liu, G., Li, C., Zhao, J., Zhang, C., & Guo, M. (2024c). ClusterKV: manipulating LLM KV cache in semantic space for recallable compression. In Proceedings of the 62nd ACM/IEEE Design Automation Conference (DAC).

Liu, J., et al. (2025a). A comprehensive survey on long context language modeling. arXiv preprint arXiv:2503.17407.

Liu, X., et al. (2025b). ChunkKV: semantic-preserving KV cache compression for efficient long-context LLM inference. arXiv preprint arXiv:2502.00299.

Liu, Y., et al. (2024d). CacheGen: KV cache compression and streaming for fast large language model serving. In Proceedings of the ACM SIGCOMM 2024 Conference (pp. 38-56).

Liu, Y., Li, X., & Wang, Z. (2025c). KV cache compression for inference efficiency in LLMs: a review. In Proceedings of the 4th International Conference on Artificial Intelligence and Intelligent Information Processing (AIIIP).

Liu, Z., et al. (2023a). Scissorhands: exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In Advances in Neural Information Processing Systems (NeurIPS) 36.

Liu, Z., et al. (2023b). Deja Vu: contextual sparsity for efficient LLMs at inference time. In Proceedings of the 40th International Conference on Machine Learning (ICML) (pp. 22137-22176).

Liu, Z., et al. (2024e). SpinQuant: LLM quantization with learned rotations. arXiv preprint arXiv:2405.16406.

Liu, Z., et al. (2024f). KIVI: a tuning-free asymmetric 2bit quantization for KV cache. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Luo, Z., et al. (2025). Shadow in the cache: unveiling and mitigating privacy risks of KV-cache in LLM inference. arXiv preprint arXiv:2508.09442.

Ma, S., et al. (2024). The era of 1-bit LLMs: all large language models are in 1.58 bits. arXiv preprint arXiv:2402.17764.

Ma, X., et al. (2026). OrbitFlow: SLO-aware long-context LLM serving with fine-grained KV cache reconfiguration. arXiv preprint arXiv:2601.10729.

Meng, F., Tang, P., Yao, Z., & Zhang, M. (2025). TransMLA: multi-head latent attention is all you need. arXiv preprint arXiv:2502.07864.

Miao, X., et al. (2023). Towards efficient generative large language model serving: a survey from algorithms to systems. arXiv preprint arXiv:2312.15234.

Mitchell, M., et al. (2019). Model cards for model reporting. In Proceedings of the Conference on Fairness, Accountability, and Transparency (FAccT) (pp. 220-229).

Modarressi, A., et al. (2025). NoLiMa: long-context evaluation beyond literal matching. In Proceedings of the 42nd International Conference on Machine Learning (ICML).

Munkhdalai, T., Faruqui, M., & Gopal, S. (2024). Leave no context behind: efficient infinite context transformers with Infini-attention. arXiv preprint arXiv:2404.07143.

Nawrot, P., et al. (2024). Dynamic memory compression: retrofitting LLMs for accelerated inference. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Oren, M., Hassid, M., Yarden, N., Adi, Y., & Schwartz, R. (2024). Transformers are multi-state RNNs (TOVA). In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP).

Pan, Z., et al. (2025). KVFlow: efficient prefix caching for accelerating LLM-based multi-agent workflows. arXiv preprint arXiv:2507.07400.

Pang, Z., et al. (2024). Cache partitioning for mitigating timing side-channel attacks in LLM serving systems. In 2024 6th International Conference on Frontier Technologies of Information and Computer (ICFTIC).

Park, J., et al. (2025). KeyDiff: key similarity-based KV cache eviction for long-context LLM inference in resource-constrained environments. arXiv preprint arXiv:2504.15364.

Peng, B., et al. (2023). RWKV: reinventing RNNs for the transformer era. In Findings of the Association for Computational Linguistics: EMNLP 2023 (pp. 14048-14077).

Peng, B., Quesnelle, J., Fan, H., & Shippole, E. (2024). YaRN: efficient context window extension of large language models. In International Conference on Learning Representations (ICLR).

Qin, R., et al. (2025a). Mooncake: a KVCache-centric disaggregated architecture for LLM serving. In Proceedings of the 23rd USENIX Conference on File and Storage Technologies (FAST) (pp. 155-170).

Qin, Z., et al. (2025b). CAKE: cascading and adaptive KV cache eviction with layer preferences. In International Conference on Learning Representations (ICLR).

Rehg, I. (2024). KV-Compress: paged KV-cache compression with variable compression rates per attention head. arXiv preprint arXiv:2410.00161.

Ribar, L., Chelombiev, I., Hudlass-Galley, L., Blake, C., Luschi, C., & Orr, D. (2024). SparQ attention: bandwidth-efficient LLM inference. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Salfati, S. (2026). Quantization dominates rank reduction for KV-cache compression. arXiv preprint arXiv:2604.11501.

Saxena, U., et al. (2024a). Eigen attention: attention in low-rank space for KV cache compression. In Findings of the Association for Computational Linguistics: EMNLP 2024.

Saxena, U., et al. (2024b). ResQ: mixed-precision quantization of large language models with low-rank residuals. arXiv preprint arXiv:2412.14363.

Seo, M., et al. (2024). IANUS: integrated accelerator based on NPU-PIM unified memory system. In Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS), Vol. 3 (pp. 545-560).

Shah, J., et al. (2024). FlashAttention-3: fast and accurate attention with asynchrony and low-precision. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Shazeer, N. (2019). Fast transformer decoding: one write-head is all you need. arXiv preprint arXiv:1911.02150.

Shen, Y., et al. (2025). LAVa: layer-wise KV cache eviction with dynamic budget allocation. arXiv preprint arXiv:2509.09754.

Sheng, Y., et al. (2023). FlexGen: high-throughput generative inference of large language models with a single GPU. In Proceedings of the 40th International Conference on Machine Learning (ICML) (pp. 31094-31116).

Sheng, Y., et al. (2024). S-LoRA: serving thousands of concurrent LoRA adapters. In Proceedings of Machine Learning and Systems (MLSys) 6.

Shi, L., Zhang, H., Yao, Y., Li, Z., & Zhao, H. (2024). Keep the cost down: a review on methods to optimize LLM's KV-cache consumption. arXiv preprint arXiv:2407.18003.

Singhania, P., Singh, S., He, S., Feizi, S., & Bhatele, A. (2024). Loki: low-rank keys for efficient sparse attention. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Song, L., et al. (2024). The early bird catches the leak: unveiling timing side channels in LLM serving systems. IEEE Transactions on Information Forensics and Security.

Strati, F., et al. (2024). DejaVu: KV-cache streaming for fast, fault-tolerant generative LLM serving. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Su, J., Ahmed, M., Lu, Y., Pan, S., Wen, B., & Liu, Y. (2024). RoFormer: enhanced transformer with rotary position embedding. Neurocomputing, 568, 127063.

Su, Z., et al. (2025). RotateKV: accurate and robust 2-bit KV cache quantization for LLMs via outlier-aware adaptive rotations. arXiv preprint arXiv:2501.16383.

Sun, Y., et al. (2023). Retentive network: a successor to transformer for large language models. arXiv preprint arXiv:2307.08621.

Sun, Y., et al. (2024). You only cache once: decoder-decoder architectures for language models. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Sun, Y., et al. (2025). Efficient attention mechanisms for large language models: a survey. arXiv preprint arXiv:2507.19595.

Tang, H., et al. (2024a). RazorAttention: efficient KV cache compression through retrieval heads. In International Conference on Learning Representations (ICLR 2025); arXiv:2407.15891.

Tang, J., et al. (2024b). Quest: query-aware sparsity for efficient long-context LLM inference. In Proceedings of the 41st International Conference on Machine Learning (ICML).

Tu, D., et al. (2024). VL-Cache: sparsity- and modality-aware KV cache compression for vision-language model inference acceleration. arXiv preprint arXiv:2410.23317.

Vaswani, A., et al. (2017). Attention is all you need. In Advances in Neural Information Processing Systems (NeurIPS) 30 (pp. 5998-6008).

Wan, Z., et al. (2024). Efficient large language models: a survey. Transactions on Machine Learning Research (TMLR).

Wang, G., et al. (2025a). LLMs know what to drop: self-attention guided KV cache eviction for efficient long-context inference (SAGE-KV). arXiv preprint arXiv:2503.08879.

Wang, H., et al. (2023). BitNet: scaling 1-bit transformers for large language models. arXiv preprint arXiv:2310.11453.

Wang, J., et al. (2025b). KVCache cache in the wild: characterizing and optimizing KVCache cache at a large cloud provider. arXiv preprint arXiv:2506.02634.

Wang, L., et al. (2026a). OptiLeak: efficient prompt reconstruction via reinforcement learning in multi-tenant LLM services. arXiv preprint arXiv:2602.20595.

Wang, T., et al. (2026b). Rethinking latency denial-of-service: attacking the LLM serving framework, not the model. arXiv preprint arXiv:2602.07878.

Wang, Z., et al. (2026c). Crystal-KV: efficient KV cache management for chain-of-thought LLMs via answer-first principle. arXiv preprint arXiv:2601.16986.

Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. Communications of the ACM, 52(4), 65-76.

Wolters, C., Yang, X., Schlichtmann, U., & Suzumura, T. (2024). Memory is all you need: an overview of compute-in-memory architectures for accelerating large language model inference. arXiv preprint arXiv:2406.08413.

Wu, G., et al. (2025). I know what you asked: prompt leakage via KV-cache sharing in multi-tenant LLM serving. In Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS).

Wu, H., & Tu, K. (2024a). A systematic study of cross-layer KV sharing for efficient LLM inference. arXiv preprint arXiv:2410.14442.

Wu, H., & Tu, K. (2024b). Layer-condensed KV cache for efficient inference of large language models. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL) (pp. 11175-11188).

Wulf, W. A., & McKee, S. A. (1995). Hitting the memory wall: implications of the obvious. ACM SIGARCH Computer Architecture News, 23(1), 20-24.

Xiao, C., et al. (2024a). InfLLM: training-free long-context extrapolation for LLMs with an efficient context memory. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Xiao, G., et al. (2024b). DuoAttention: efficient long-context LLM inference with retrieval and streaming heads. In International Conference on Learning Representations (ICLR 2025); arXiv:2410.10819.

Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2023). SmoothQuant: accurate and efficient post-training quantization for large language models. In Proceedings of the 40th International Conference on Machine Learning (ICML) (pp. 38087-38099).

Xiao, G., Tian, Y., Chen, B., Han, S., & Lewis, M. (2024c). Efficient streaming language models with attention sinks. In International Conference on Learning Representations (ICLR).

Xie, Z., et al. (2025). Strata: hierarchical context caching for long context language model serving. arXiv preprint arXiv:2508.18572.

Xu, Y., Khaira, N. K., & Singh, T. (2026). KV cache optimization strategies for scalable and efficient LLM inference. arXiv preprint arXiv:2603.20397.

Yamamoto, Y., et al. (2026). Bit-flip vulnerability of shared KV-cache blocks in LLM serving systems. arXiv preprint.

Yan, S., et al. (2025). Memory-R1: enhancing large language model agents to manage and utilize memories via reinforcement learning. arXiv preprint arXiv:2508.19828.

Yang, D., Han, X., Gao, Y., Hu, Y., Zhang, S., & Zhao, H. (2024a). PyramidInfer: pyramid KV cache compression for high-throughput LLM inference. In Findings of the Association for Computational Linguistics: ACL 2024 (pp. 3258-3270).

Yang, H., et al. (2025a). KVShare: an LLM service system with efficient and effective multi-tenant KV cache reuse. arXiv preprint arXiv:2503.16525.

Yang, J., et al. (2026). Rethinking KV cache eviction via a unified information-theoretic objective (CapKV). arXiv preprint arXiv:2604.25975.

Yang, J., Hou, B., Wei, W., Bao, Y., & Chang, S. (2025b). KVLink: accelerating large language models via efficient KV cache reuse. arXiv preprint arXiv:2502.16002.

Yang, L., Zhang, Z., Chen, Z., Li, Z., & Jia, Z. (2024b). TidalDecode: fast and accurate LLM decoding with position persistent sparse attention. arXiv preprint arXiv:2410.05076.

Yang, S., Sheng, Y., Gonzalez, J. E., Stoica, I., & Zheng, L. (2024c). Post-training sparse attention with double sparsity. arXiv preprint arXiv:2408.07092.

Yang, X., Yan, B., Li, H., & Chen, Y. (2020). ReTransformer: ReRAM-based processing-in-memory architecture for transformer acceleration. In Proceedings of the 39th International Conference on Computer-Aided Design (ICCAD).

Yao, H., et al. (2026). Thin keys, full values: reducing KV cache via low-dimensional attention selection. arXiv preprint arXiv:2603.04427.

Yao, J., et al. (2024). CacheBlend: fast large language model serving for RAG with cached knowledge fusion. In Proceedings of the Twentieth European Conference on Computer Systems (EuroSys 2025).

Ye, H., et al. (2025a). KVCOMM: online cross-context KV-cache communication for efficient LLM-based multi-agent systems. arXiv preprint arXiv:2510.05366.

Ye, J., et al. (2026). MemWeaver: weaving hybrid memories for traceable long-horizon agentic reasoning. arXiv preprint arXiv:2601.18204.

Ye, L., Tao, Z., Huang, Y., & Li, Y. (2024). ChunkAttention: efficient self-attention with prefix-aware KV cache and two-phase partition. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL) (pp. 11608-11620).

Ye, Z., et al. (2025b). FlashInfer: efficient and customizable attention engine for LLM inference serving. In Proceedings of Machine Learning and Systems (MLSys) 7.

Yen, H., et al. (2025). HELMET: how to evaluate long-context language models effectively and thoroughly. In International Conference on Learning Representations (ICLR).

Yu, G.-I., Jeong, J. S., Kim, G.-W., Kim, S., & Chun, B.-G. (2022). Orca: a distributed serving system for transformer-based generative models. In Proceedings of the 16th USENIX Symposium on Operating Systems Design and Implementation (OSDI) (pp. 521-538).

Yu, H., et al. (2024). Effectively compress KV heads for LLM. arXiv preprint arXiv:2406.07056.

Yuan, J., et al. (2025). Native sparse attention: hardware-aligned and natively trainable sparse attention. arXiv preprint arXiv:2502.11089.

Yuan, T., et al. (2024a). LV-Eval: a balanced long-context benchmark with 5 length levels up to 256K. arXiv preprint arXiv:2402.05136.

Yuan, Z., et al. (2023). ASVD: activation-aware singular value decomposition for compressing large language models. arXiv preprint arXiv:2312.05821.

Yuan, Z., et al. (2024b). LLM inference unveiled: survey and roofline model insights. arXiv preprint arXiv:2402.16363.

Yue, Y., Yuan, Z., Duanmu, H., Zhou, S., Wu, J., & Nie, L. (2024). WKVQuant: quantizing weight and key/value cache for large language models gains more. arXiv preprint arXiv:2402.12065.

Zeng, Z., et al. (2025). SubKV: quantizing long context KV cache for sub-billion parameter language models on edge devices. Software: Practice and Experience, 55(8), 1287-1304.

Zhang, H., et al. (2026a). HERMES: KV cache as hierarchical memory for efficient streaming video understanding. arXiv preprint arXiv:2601.14724.

Zhang, P., et al. (2024a). Long context compression with activation beacon. arXiv preprint arXiv:2401.03462.

Zhang, R., et al. (2024b). LoRC: low-rank compression for LLMs KV cache with a progressive compression strategy. arXiv preprint arXiv:2410.03111.

Zhang, T., Yi, J., Xu, Z., & Shrivastava, A. (2024c). KV cache is 1 bit per channel: efficient large language model inference with coupled quantization. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Zhang, X., et al. (2024d). Infinity-Bench: extending long context evaluation beyond 100K tokens. In Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL) (pp. 15262-15277).

Zhang, X., et al. (2026b). ActMem: bridging the gap between memory retrieval and reasoning in LLM agents. arXiv preprint arXiv:2603.00026.

Zhang, Y., et al. (2025). Memory-as-action: autonomous context curation for long-horizon agentic tasks. arXiv preprint arXiv:2510.12635.

Zhang, Z., et al. (2023). H2O: heavy-hitter oracle for efficient generative inference of large language models. In Advances in Neural Information Processing Systems (NeurIPS) 36.

Zhang, Z., et al. (2024e). A survey on the memory mechanism of large language model based agents. ACM Transactions on Information Systems.

Zhang, Z., et al. (2024f). Q-Hitter: a better token oracle for efficient LLM inference via sparse-quantized KV cache. In Proceedings of Machine Learning and Systems (MLSys) 6.

Zhao, Y., et al. (2024). Atom: low-bit quantization for efficient and accurate LLM serving. In Proceedings of Machine Learning and Systems (MLSys) 6.

Zhao, Y., et al. (2026). AMA-Bench: evaluating long-horizon memory for agentic applications. arXiv preprint arXiv:2602.22769.

Zhen, R., et al. (2025). Taming the titans: a survey of efficient LLM inference serving. arXiv preprint arXiv:2504.19720.

Zheng, L., et al. (2024a). SGLang: efficient execution of structured language model programs. In Advances in Neural Information Processing Systems (NeurIPS) 37.

Zheng, X., et al. (2024b). InputSnatch: stealing input in LLM services via timing side-channel attacks. arXiv preprint arXiv:2411.18191.

Zhong, Y., et al. (2024). DistServe: disaggregating prefill and decoding for goodput-optimized large language model serving. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI) (pp. 193-210).

Zhou, X., et al. (2024a). DynamicKV: task-aware adaptive KV cache compression for long context LLMs. arXiv preprint arXiv:2412.14838.

Zhou, Y., et al. (2025a). EliteKV: scalable KV cache compression via RoPE frequency selection and joint low-rank projection. arXiv preprint arXiv:2503.01586.

Zhou, Z., et al. (2024b). A survey on efficient inference for large language models. arXiv preprint arXiv:2404.14294.

Zhou, Z., et al. (2025b). MEM1: learning to synergize memory and reasoning for efficient long-horizon agents. arXiv preprint arXiv:2506.15841.